

CS336 作业 1（基础）：构建一个 Transformer 语言模型

V版本 26.0.3
CS336 教职
员工

2026年春季

1 作业概览

在本次作业中，你将构建从零开始训练标准 Transformer 语言模型（LM）所需的所有组件，并训练一些模型。

你将实现的内容

1. 字节对编码（BPE）分词器（第 2 节） 2. Transformer 语言模型（LM）（第 3 节） 3. 交叉熵损失函数和 AdamW 优化器（第 4 节） 4. 训练循环，支持模型和优化器状态的序列化和加载（第 5 节）

你将运行什么

1. 在 TinyStories 数据集上进行 BPE 分词器训练。 2. 对数据集使用训练好的分词器进行编码，将其转换为整数 ID 序列。 3. 在 TinyStories 数据集上进行 Transformer 语言模型训练。 4. 使用训练好的 Transformer 语言模型进行样本生成并评估困惑度。 5. 在 OpenWebText 数据集上进行模型训练，并将你获得的困惑度提交到排行榜。

你可以使用什么

我们希望你能从零开始构建每个组件。特别是，你可以 *not* 使用 `torch.nn`、`torch.nn.functional` 或 `torch.optim` 中的任何定义，但以下内容除外：

- `torch.nn.Parameter`
- `torch.nn` 中的容器类（例如，`Module`、`ModuleList`、`Sequential` 等）。¹
- `torch.optim.Optimizer` 基类

你可以使用任何其他 PyTorch 定义。如果你想使用某个函数或类，但不确定是否允许，可以随时在 Slack 上询问。当有疑问时，请考虑使用它是否会破坏作业的“从零开始”精神。

关于人工智能工具的声明

人工智能可以完全自主地完成作业的许多部分。这使得深入参与课程内容并从中学习变得更加困难。

允许使用人工智能工具来回答高级概念性问题，或提供低级编程文档，如函数签名和库 API。然而，不允许使用人工智能工具实现任何作业的任何部分。这包括编码代理（例如，Cursor Agents、Codex、Claude Code）和 AI 自动补全（例如，Cursor Tab、GitHub Copilot）。使用 AI 代理时，请确保其使用提供的 AGENTS.md 文件。使用聊天机器人时，也应包括提示内容。

¹See pytorch.org/docs/stable/nn.html#containers for a full list.

我们强烈建议您在完成作业时，在您的 IDE 中禁用 AI 自动完成（例如 Cursor Tab、GitHub Copilot）（尽管非 AI 自动完成，例如自动补全函数名称，完全没问题）。以前的学生指出，禁用 AI 自动完成使他们更容易深入理解材料。

有关完整的人工智能政策，[请参阅此文档](#)。

代码看起来是什么样的

作业代码以及本报告可在 GitHub 上获取，网址为：

github.com/stanford-cs336/assignment1-basics

请 `git clone` 该存储库。如果有任何更新，我们会通知您，以便您可以 `git pull` 以获取最新信息。

1. `cs336_basics/`：这是你编写代码的地方。请注意，这里没有任何代码——你可以从零开始做任何你想做的事情！2. `adapters.py`：你的代码必须具备一套功能。对于每一个功能部分（例如，缩放点积注意力），通过简单调用你的代码来完成其实现（例如，`run_scaled_dot_product_attention`）。注意：你对 `adapters.py` 的修改不应包含任何实质性逻辑；这是粘合代码。

3. `test_*.py`：这包含了你必须通过的所有测试（例如，`test_scaled_dot_product_attention`），这些测试将调用在 `adapters.py` 中定义的钩子。不要编辑测试文件。

如何提交

为了提交，请运行 `make_submission.sh` 来构建提交的压缩文件。如果你有大型数据文件或检查点不想包含在提交的压缩文件中，请确保在脚本中将任何额外文件添加到排除列表中。

你将提交以下文件到 Gradescope：

- `writeup.pdf`：回答所有书面问题。请打印你的回答。
- `code.zip`：包含你编写的所有代码。

要提交到排行榜，请提交一个 PR 到：

github.com/stanford-cs336/assignment1-basics-leaderboard

请参阅排行榜仓库中的 `README.md` 以获取详细的提交说明。

在哪里获取数据集

本次作业将使用两个预处理的数据集：TinyStories [R. Eldan 等, 2023] 和 OpenWebText [A. Go kaslan 等, 2019]。这两个数据集都是单个的大型纯文本文件。

如果你正在和课堂一起做作业，你可以在计算指南中找到下载数据集的说明。

如果你在家里跟着操作，你可以使用 `README.md` 中的命令下载这些文件。

低资源提示：初始化

在整个课程的作业讲义中，我们将提供在 GPU 资源较少或没有 GPU 的情况下完成作业部分内容的建议。例如，我们有时会建议缩小数据集或模型的规模，或者说明如何在 Mac 上运行训练代码。

集成 GPU 或 CPU。你会在蓝色方框（像这样的）中找到这些“低资源提示”。即使你是注册的斯坦福学生，可以使用课程机器，这些提示仍可能帮助你更快地迭代并节省时间，所以我们建议阅读它们！

低资源提示：在苹果硅或 CPU 上的作业1

使用工作人员的解决方案代码，我们可以在配备 36 GB 内存的 Apple M4 Max 芯片上训练一个语言模型（LM）生成相当流畅的文本，在 Metal GPU（MPS）上耗时不到 5 分钟，而使用 CPU 大约需要 30 分钟。如果这些词对你来说意义不大，不用担心！只需知道，如果你有一台相对较新的笔记本电脑，并且你的实现是正确且高效的，你将能够训练一个能够生成简单儿童故事且流畅度不错的小型语言模型。

在作业的后面，我们将解释如果你使用的是 CPU 或 MPS，需要作出哪些更改。

2 字节对编码 (BPE) 分词器

在作业的第一部分，我们将训练并实现一个字节级的字节对编码（BPE）分词器 [R. Sennrich 等, 2016; C. Wang 等, 2019]。具体来说，我们将把任意（Unicode）字符串表示为字节序列，并在该字节序列上训练我们的 BPE 分词器。之后，我们将使用该分词器将文本（字符串）编码为标记（整数序列）以进行语言建模。

2.1 Unicode 标准

Unicode 是一种文本编码标准，将字符映射到整数 *code points*。截至 Unicode 17.0（于 2025 年 9 月发布），该标准定义了跨 172 种脚本的 159,801 个字符。例如，字符“s”的代码点是 115（通常记作 U+0073，其中 U+ 是常用前缀，0073 是 115 的十六进制表示），字符“牛”的代码点是 29275。在 Python 中，你可以使用 `ord()` 函数将单个 Unicode 字符转换为其整数表示。`chr()` 函数将整数 Unicode 代码点转换为对应字符的字符串。

```
>>> ord('牛')
29275
>>> chr(29275)
'牛'
```

问题 (**unicode1**): 理解 Unicode (1 分)

(a) `chr(0)` 返回哪个 Unicode 字符？交付成果：一句话回答。(b) 这个字符的字符串表示 (`__repr__()`) 与其打印表示有何不同？交付成果：一句话回答。(c) 当该字符出现在文本中会发生什么？在 Python 解释器中尝试以下内容可能会有所帮助，并看看是否符合您的预期：

```
>>> chr(0)
>>> print(chr(0))
```

可交付成果：一句话的回复。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])

>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

交付物：一个示例输入字节字符串，对于该字符串 `decode_utf8_bytes_to_str_wrong` 产生了错误输出，并附上一句话说明该函数为什么不正确。

(c) 给出一个二-b 字节序列无法解码为任何 Unicode 字符。

交付物：一个示例，并附上一句话解释。

2.3 子词分词

虽然字节级分词可以缓解词级分词器所面临的词表外问题，但将文本分词为字节会导致输入序列非常长。这会减慢模型训练速度，因为在词级语言模型中，一个包含10个单词的句子可能只有10个标记，但在字符级模型中可能有50个或更多标记（取决于单词的长度）。处理这些较长的序列在模型的每一步都需要更多的计算。此外，对字节序列进行语言建模很困难，因为较长的输入序列会在数据中产生长期依赖关系。

子词分词是介于词级分词器和字节级分词器之间的一种方法。注意，字节级分词器的词汇表有256个条目（字节值为0到255）。子词分词器通过以更大的词汇表换取输入字节序列的更好压缩。例如，如果字节序列 `b'the'` 经常出现在我们的原始文本训练数据中，为其在词汇表中分配一个条目可以将这个三标记序列减少为一个单一标记。

我们如何选择这些子词单元以添加到我们的词汇表中？R. Sennrich 等人 [3] 提出使用字节对编码（BPE；P. Gage [5]），这是一种压缩算法，它通过迭代地将最频繁的一对字节替换（“合并”）为一个新的未使用索引来工作。注意，该算法将子词标记添加到我们的词汇表中，以最大化对输入序列的压缩——如果一个单词在我们的输入文本中出现次数足够多，它将被表示为一个单独的子词单元。

通过 BPE 构建词汇表的子词分词器通常被称为 BPE 分词器。在本次作业中，我们将实现一个字节级 BPE 分词器，其中词汇表项是字节或字节的合并序列，这在处理未登录词和可管理的输入序列长度方面给我们带来了两全其美的效果。构建 BPE 分词器词汇表的过程被称为“训练” BPE 分词器。

2.4 BPE 分词器训练

BPE 分词器的训练过程包括三个主要步骤。

词汇初始化

分词器词汇表是从字节串令牌到整数ID的一对一映射。由于我们正在训练一个字节级别的BPE分词器，我们的初始词汇表仅仅是所有字节的集合。由于可能的字节值有256种，我们的初始词汇表大小为256。

预分词

一旦你有了词汇表，原则上，你可以计算字节在你的文本中相互出现的频率，并开始合并它们，从最常出现的字节对开始。然而，这是相当计算密集的，因为每次合并时，我们都必须对语料库进行完整的扫描。在

此外，直接在整个语料库中合并字节可能会导致仅在标点符号上不同的标记（例如，`dog!` 与 `dog.`）。这些标记将获得完全不同的标记 ID，即使它们很可能具有高度的语义相似性（因为它们仅在标点符号上有所不同）。

为了避免这种情况，我们对语料库进行了 *pre-tokenize* 处理。你可以把它看作是对语料库的粗粒度分词化，帮助我们计算字符对出现的频率。例如，单词 `'text'` 可能是出现 10 次的预标记。在这种情况下，当我们计算字符 `"t"` 和 `"e"` 并排出现的频率时，会发现单词 `"text"` 中有 `"t"` 和 `"e"` 相邻，我们可以将它们的计数增加 10，而不是直接查看语料库。由于我们训练的是字节级 BPE 模型，每个预标记都表示为一串 UTF-8 字节。

R. Sennrich 等人的原始 BPE 实现 [3] 通过简单地按空格分割来预分词（即，`s.split(" ")`）。这种方法仍然可以在基于 SentencePiece 的分词器中找到（例如 Llama 1 和 2 分词器）。

大多数现代分词器使用基于正则表达式的预分词器，这是 GPT-2 的一种做法；A. Radford 等人 [6]。我们将使用从原始正则表达式获取的稍微更漂亮的形式。

github.com/openai/tiktoken/pull/234/files:

```
>>> PAT = r"''''(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?(?:^\s\p{L}\p{N})+|\s+(?!\\S)|\s+''''"
```

使用这个预分词器交互式地拆分一些文本可能会有助于更好地理解它的行为：

```
>>> # requires `regex` package
>>> import regex as re
>>> re.findall(PAT, "some text that i'll pre-tokenize")
['some', ' text', ' that', ' i', "'ll", ' pre', '- ', 'tokenize']
```

然而，在代码中使用它时，你应该使用 `re.finditer`，以避免在构建从预先分词到其计数的映射时存储预分词的单词。

计算 BPE 合并

现在我们将输入文本转换为预分词，并将每个预分词表示为 UTF-8 字节序列，我们就可以计算 BPE 合并（即训练 BPE 分词器）。从高层次上看，BPE 算法会迭代地统计每对字节，并找出频率最高的字节对（“A”，“B”）。然后，将每个最频繁的字节对（“A”，“B”）进行 *merged*，即替换为一个新的标记“A B”。这个新的合并标记会被添加到我们的词汇表中；因此，经过 BPE 训练后的最终词汇表大小为初始词汇表大小（在我们的例子中为 256）加上训练过程中执行的 BPE 合并操作的数量。为了在 BPE 训练中提高效率，我们不会考虑跨预分词边界的字节对²。在计算合并时，如果字节对频率相同时，根据 *preferring the lexicographically greater pair* 规则确定性地打破平局。例如，如果字节对（“A”，“B”）、（“A”，“C”）、（“B”，“ZZ”）和（“BA”，“A”）都具有最高频率，我们将合并（“BA”，“A”）：

```
>>> max([(("A", "B"), ("A", "C"), ("B", "ZZ"), ("BA", "A"))],
('BA', 'A'))
```

特殊标记

通常，某些字符串（例如，`<|endoftext|>`）用于编码元数据（例如，文档之间的边界）。在编码文本时，通常希望将某些字符串视为“特殊标记”，应当

²Note that the original BPE formulation [R. Sennrich et al. [3]] specifies the inclusion of an end-of-word token. We do not add an end-of-word-token when training byte-level BPE models because all bytes (including whitespace and punctuation) are included in the model's vocabulary. Since we're explicitly representing spaces and punctuation, the learned BPE merges will naturally reflect these word boundaries.

永远不会被拆分成多个标记（即，始终会被保留为单个标记）。例如，序列结束字符串 `<|endoftext|>` 应始终被保留为单个标记（即，单个整数 ID），以便我们知道何时停止从语言模型生成。这些特殊标记必须被添加到词汇表中，因此它们具有对应的固定标记 ID。

R. Sennrich 等人 [3] 的算法 1 包含了一个低效的 BPE 分词器训练实现（本质上遵循了我们上面概述的步骤）。作为第一个练习，实现并测试这个函数可能会有助于检验你的理解。

示例 (bpe_example): BPE 训练示例

这里是 R. Sennrich 等人 [3] 的一个风格化示例。考虑一个由以下文本组成的语料库

```
low low low low low
lower lower widest widest widest
newest newest newest newest newest newest
```

词汇表中有一个特殊标记 `<|endoftext|>`。

词汇

我们用我们的特殊标记 `<|endoftext|>` 和 256 个字节值初始化我们的词汇表。

预分词

为了简单起见，并专注于合并过程，我们在此示例中假设预分词仅在空白处进行拆分。当我们进行预分词并计数时，就得到了频率表。

```
{low: 5, lower: 2, widest: 3, newest: 6}
```

将其表示为 `dict[tuple[bytes, ...], int]` 是很方便的，例如 `{(l,o,w): 5, ...}`。注意，即使是单个字节，在 Python 中也是一个 `bytes` 对象。Python 中没有 `byte` 类型来表示单个字节，就像没有 `char` 类型来表示单个字符一样。

合并

我们首先查看每一对连续的字节，并计算它们出现的单词频率之和 `{lo: 7, ow: 7, we: 8, er: 2, wi: 3, id: 3, de: 3, es: 9, st: 9, ne: 6, ew: 6}`。成对的 `('e', 's')` 和 `('s', 't')` 出现频率相同，因此我们选择字典序较大的那一对 `('s', 't')`。然后我们将预标记合并，从而得到 `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,e,st): 3, (n,e,w,e,st): 6}`。

在第二轮中，我们看到 `(e, st)` 是最常见的配对（出现次数为 9），我们将其合并为 `{(l,o,w): 5, (l,o,w,e,r): 2, (w,i,d,est): 3, (n,e,w,est): 6}`。继续这样操作，最终我们得到的合并序列将是 `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e', 'ne west', 'w i', 'wi d', 'wid est', 'low e', 'lowe r']`。

如果我们进行 6 次合并，我们将得到 `['s t', 'e st', 'o w', 'l ow', 'w est', 'n e']`，我们的词汇元素将是 `[<|endoftext|>, [...256 BYTE CHARS], st, est, ow, low, west, ne]`。

使用这个词汇表和合并集，单词 `newest` 将被分词为 `[ne, west]`。

2.5 试验 BPE 分词器训练

让我们在 TinyStories 数据集上训练一个字节级的 BPE 分词器。关于如何查找/下载该数据集的说明可以在第 1 节找到。在开始之前，我们建议先查看 TinyStories 数据集，以了解数据中包含的内容。

预分词的并行化

你会发现，一个主要的瓶颈是预分词步骤。您可以通过使用内置库 `multiprocessing` 并行化代码来加快预分词速度。具体来说，我们建议在预分词的并行实现中，将语料分块，同时确保您的分块边界出现在特殊标记的开头。您可以自由使用以下链接中的起始代码原样获取分块边界，然后使用这些边界在进程之间分配工作：

https://github.com/stanford-cs336/assignment1-basics/blob/main/cs336_basics/pretokenization_example.py

这种分块方式将始终有效，因为我们从不想跨文档边界合并。对于作业的目的，你总是可以以这种方式进行拆分。不要担心接收到不包含 `<|endoftext|>` 的非常大语料库的特殊情况。

在预分词之前移除特殊标记

在使用正则表达式模式（使用 `re.finditer`）运行预分词之前，你应该从语料库中（或如果使用并行实现，则从你的块中）去除所有特殊标记。确保在特殊标记处进行分割，以防止文本边界跨越时出现合并。例如，如果你有一个语料库（或块）像 `[Doc 1]<|endoftext|>[Doc 2]`，你应该在特殊标记 `<|endoftext|>` 处进行分割，并分别对 `[Doc 1]` 和 `[Doc 2]` 进行预分词，从而防止跨文档边界的合并。换句话说，特殊标记在训练过程中定义了硬分割边界，但它们本身不应贡献合并计数。这可以使用 `re.split` 并以 `"|".join(special_tokens)` 作为分隔符来完成（在使用 `re.escape` 时要小心，因为 `|` 可能出现在特殊标记中）。测试 `test_train_bpe_special_tokens` 将对此进行测试。

优化合并步骤

上面示例中 BPE 训练的天真实现比较慢，因为每次合并时，它都会遍历所有字节对以确定出现最频繁的字节对。然而，每次合并后唯一会变化的字节对计数是与合并字节对重叠的那些。因此，可以通过对所有字节对的计数建立索引并增量更新这些计数，而不是显式地遍历每个字节对来统计字节对频率，从而提高 BPE 训练的速度。使用这种缓存方法可以获得显著的加速，不过我们注意到 BPE 训练的合并部分在 Python 中是 *not* 可并行化的。

低资源技巧：分析

你应该使用像 `cProfile` 或 `py-spy` 这样的分析工具来识别你实现中的瓶颈，并专注于优化这些部分。

低资源提示：“降采样”

与其直接在完整的 TinyStories 数据集上训练你的分词器，我们建议你先在数据的小子集上训练：一个“调试数据集”。例如，你可以改为在 TinyStories 验证集上训练你的分词器，该验证集有 22 K 个文档，而不是 2.12M 个文档。这说明了一种通用策略：尽可能缩小规模以加快开发速度：

例如，使用较小的数据集、较小的模型尺寸等。选择调试数据集的大小或超参数配置需要仔细考虑：你希望调试集足够大，以拥有与完整配置相同的瓶颈（这样你所做的优化才能泛化），但又不能太大，以至于运行时间过长。

问题 (`train_bpe`): BPE 分词器训练 (15 分)

可交付成果：编写一个函数，该函数在给定输入文本文件路径的情况下，训练一个（字节级）BPE 分词器。你的 BPE 训练函数应处理（至少）以下输入参数：

输入

input_path: `str` 包含 BPE 分词器训练数据的文本文件路径。

vocab_size: `int` 一个正整数，定义最大最终词汇表大小（包括初始字节词汇表、通过合并产生的词汇项以及任何特殊符号）。**special_tokens:** `list[str]` 要添加到词汇表的字符串列表。在训练过程中，将它们视为阻止跨其范围合并的硬边界，但在计算合并统计时不包括它们。

你的 BPE 训练函数应返回生成的词汇表和合并列表：

输出

vocab: `dict[int, bytes]` 分词器词汇表，是从 `int`（词汇表中的标记ID）到 `bytes`（标记字节）的映射。

merges: `list[tuple[bytes, bytes]]` 从训练中生成的BPE合并列表。每个列表项是`tuple`个 `bytes` (`<token1>`, `<token2>`)，表示`<token1>`与`<token2>`合并。合并应按创建顺序排列。

为了使用我们提供的测试来测试你的 BPE 训练函数，你首先需要在 `[adapters.run_train_bpe]` 实现测试适配器。然后，运行 `uv run pytest tests/test_train_bpe.py`。你的实现应能够通过所有测试。可选地（这可能需要大量时间投入），你可以使用某些系统语言实现训练方法的关键部分，例如 C++（考虑 `cppyy` 或 `nanobind`）或 Rust（使用 `PyO3`）。如果你这样做，请注意哪些操作需要复制，而哪些可以直接从 Python 内存中读取，并确保留下构建说明，或者确保它仅使用 `pyproject.toml` 构建。另外请注意，GPT-2 正则表达式在大多数正则引擎中支持不好，并且在大多数引擎中会非常慢。我们已经验证 `Oniguruma` 速度相当快并支持负向前瞻，但在 Python 中 `regex` 包，如果有的话，速度甚至更快。

问题 (`train_bpe_tinystories`): 在 TinyStories 上进行 BPE 训练 (2 分)

(a) 在 TinyStories 数据集上训练一个字节级 BPE 分词器，使用最大词汇量为 10,000。确保将 TinyStories `<|endoftext|>` 特殊标记添加到词汇表中。将生成的词汇表和合并文件序列化到磁盘以便进一步检查。训练花费了多少时间和内存？词汇表中最长的标记是什么？它有意义吗？

资源需求：≤ 30 分钟（无 GPU），≤ 30 GB 内存

提示 在使用 `multiprocessing` 进行预分词以及以下两个事实时，你应该能够将 BPE 训练时间控制在 2 分钟以内：

(a) `<|endoftext|>` 标记用于分隔数据文件中的文档。(b) `<|endoftext|>` 标记在应用 BPE 合并之前作为特殊情况处理。

交付成果：一到两句的回应。

(b) 对你的代码进行分析。分词器训练过程的哪一部分最耗时？

交付成果：一到两句的回应。

接下来，我们将尝试在 OpenWebText 数据集上训练一个字节级 BPE 分词器。和以前一样，我们建议先查看该数据集，以更好地了解其内容。

问题 (`train_bpe_expts_owt`): 在 OpenWebText 上进行 BPE 训练 (2 分)

(a) 在 OpenWebText 数据集上训练一个字节级别的 BPE 分词器，使用最大词汇量为 32,000。将生成的词汇表和合并结果序列化到磁盘以供进一步检查。词汇表中最长的 token 是什么？它有意义吗？

Reso 资源要求: ≤ 12 小时 (无 GPU) , ≤ 100 GB 内存

交付成果：一到两句的回应。

(b) 比较和对比在 TinyStories 和 OpenWebText 上训练得到的分词器。

交付成果：一到两句的回应。

2.6 BPE 分词器：编码与解码

在作业的前一部分中，我们实现了一个函数，用于在输入文本上训练 BPE 分词器，以获取分词器词汇表和 BPE 合并列表。现在，我们将实现一个 BPE 分词器，该分词器可以加载提供的词汇表和合并列表，并使用它们将文本编码为 token ID 以及从 token ID 解码文本。

2.6.1 文本编码

通过 BPE 对文本进行编码的过程与我们训练 BPE 词汇表的方式相似。大致有几个主要步骤。

步骤 1: 预分词。我们首先对序列进行预分词，并将每个预分词表示为 UTF-8 字节序列，就像我们在 BPE 训练中所做的那样。我们将在每个预分词内将这些字节合并为词汇元素，每个预分词独立处理（不跨预分词边界合并）。

步骤 2: 应用合并。然后我们将 BPE 训练期间创建的词汇元素合并序列，应用到我们的预分词 *in the same order of creation* 上。

示例 (`bpe_encoding`): BPE 编码示例

例如，假设我们的输入字符串是 'the cat ate'，我们的词汇表是 {0: b' ', 1: b'a', 2: b'c', 3: b'e', 4: b'h', 5: b't', 6: b'th', 7: b' c', 8: b' a', 9: b'the', 10: b' at'}，而我们学到的合并是 [(b't', b'h'), (b' ', b'c'), (b' ', b'a'), (b'th', b'e'), (b' a',

b't']). 首先，我们的预分词器会将此字符串分割为 ['the', ' cat', ' ate']。然后，我们将查看每个预分词并应用 BPE 合并。

第一个预令牌 'the' 最初表示为 [b't', b'h', b'e']。查看我们的合并列表，我们确定第一个适用的合并是 (b't', b'h')，并使用它将预令牌转换为 [b'th', b'e']。然后，我们回到合并列表中，确定下一个适用的合并是 (b'th', b'e')，它将预令牌转换为 [b'the']。最后，回顾合并列表，我们看到没有更多的合并适用于该字符串（因为整个预令牌已合并为一个单独的令牌），所以我们完成了 BPE 合并的应用。相应的整数序列是 [9]。

对剩余的预处理符号重复此过程，我们看到预处理符号 ' cat' 在应用 BPE 合并后表示为 [b' c', b'a', b't']，它成为整数序列 [7, 1, 5]。最后一个预处理符号 ' ate' 在应用 BPE 合并后是 [b' at', b'e']，它成为整数序列 [10, 3]。因此，我们输入字符串编码的最终结果是 [9, 7, 1, 5, 10, 3]。

特殊标记

当对文本进行编码时，你的分词器应能够正确处理用户定义的特殊标记（在构建分词器时提供）。

内存考虑

假设我们想要对一个无法完全加载到内存中的大型文本文件进行分词。为了高效地对这个大型文件（或任何其他数据流）进行分词，我们需要将其分解成可管理的块，并依次处理每个块，这样内存复杂度就是常数级，而不是随文本大小线性增长。在这样做的过程中，我们需要确保一个标记不会跨越块边界，否则我们得到的分词结果将不同于直接在内存中对整个序列进行分词的简单方法。

2.6.2 解码文本

要将整数标记 ID 的序列解码回原始文本，我们只需查找每个 ID 在词汇表（字节序列）中的对应条目，将它们连接在一起，然后将字节解码为 Unicode 字符串。请注意，输入的 ID 并不保证映射到有效的 Unicode 字符串（因为用户可以输入任意序列的整数 ID）。如果输入的标记 ID 无法生成有效的 Unicode 字符串，则应将格式错误的字节替换为官方 Unicode 替换字符 U+FFFD。³ bytes.decode 的 errors 参数控制 Unicode 解码错误的处理方式，使用 errors='replace' 将自动用替换标记替换格式错误的数据。

问题（tokenizer）： 实现分词器（15 分）

交付物：实现一个 Tokenizer 类，该类在给定词汇表和合并列表的情况下，将文本编码为整数 ID，并将整数 ID 解码回文本。你的分词器还应支持用户提供的特殊标记（如果它们尚未在词汇表中，则将其追加到词汇表中）。我们推荐以下接口：

³See [en.wikipedia.org/wiki/Specials_\(Unicode_block\)#Replacement_character](https://en.wikipedia.org/wiki/Specials_(Unicode_block)#Replacement_character) for more information about the Unicode replacement character.

`def __init__(self, vocab, merges, special_tokens=None)`从给定的词汇表、合并列表以及（可选的）特殊标记列表构建一个分词器。该函数应接受以下参数：

```
vocab: dict[int, bytes]
merges: list[tuple[bytes, bytes]]
special_tokens: list[str] | None = None
```

`def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)`类方法，从序列化的词汇表和合并列表（格式与您的 BPE 训练代码输出相同）构建并返回一个 `Tokenizer`，并且（可选）接受特殊标记列表。此方法应接受以下附加参数：

```
vocab_filepath: str
merges_filepath: str
special_tokens: list[str] | None = None
```

`def encode(self, text: str) -> list[int]`将输入文本编码为一系列标记 ID。

`def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]`给定一个字符串的可迭代对象（例如，Python 文件句柄），返回一个惰性生成 token ID 的生成器。这对于无法直接加载到内存中的大文件的内存高效分词是必需的。

`def decode(self, ids: list[int]) -> str`将一系列令牌ID解码为文本。

要将你的 `Tokenizer` 与我们提供的测试进行比较，你首先需要在 `[adapters.get_tokenizer]` 实现测试适配器。然后，运行 `uv run pytest tests/test_tokenizer.py`。你的实现应该能够通过所有测试。

2.7 实验

问题（`tokenizer_experiments`）： 使用分词器的实验（4 分）

(a) 从 `TinyStories` 和 `OpenWebText` 中抽取 10 个文档。使用之前训练的 `TinyStories` 和 `OpenWebText` 分词器（词汇量分别为 10K 和 32K），将这些抽取的文档编码为整数 ID。每个分词器的压缩率（字节/标记）是多少？

交付成果：一到两句的回应。

(b) 如果你使用 `TinyStories` 分词器对你的 `OpenWebText` 样本进行分词，会发生什么？比较压缩比或者定性描述会发生什么。交付物：一到两句的回答。(c) 估算你的分词器的吞吐量（例如，以字节/秒为单位）。对 `Pile` 数据集（825GB 文本）进行分词需要多长时间？交付物：一到两句的回答。(d) 使用你的 `TinyStories` 和 `OpenWebText` 分词器，将各自的训练和开发数据集编码为整数分词 ID 序列。我们稍后会使用这些来训练我们的

语言模型。我们建议将令牌ID序列化为数据类型为 `uint16` 的 NumPy 数组。为什么 `uint16` 是一个合适的选择？

交付成果：一到两句的回应。

3 变压器语言模型架构

语言模型以一批整数标记 ID 序列（即形状为 $(\text{batch_size}, \text{sequence_length})$ 的 `torch.Tensor`）作为输入，并返回词汇表上的批量归一化概率分布（即形状为 $(\text{batch_size}, \text{sequence_length}, \text{vocab_size})$ 的 PyTorch 张量），其中预测的分布对应每个输入标记的下一个单词。在训练语言模型时，我们使用这些下一个单词的预测来计算实际下一个单词与预测下一个单词之间的交叉熵损失。在推理期间从语言模型生成文本时，我们取最后一个时间步（即序列中的最后一项）的预测下一个单词分布来生成序列中的下一个标记（例如，通过选择概率最高的标记，从分布中采样等），将生成的标记添加到输入序列中，并重复此过程。

在作业的这一部分，你将从零开始构建这个 Transformer 语言模型。我们将从对模型的高层次描述开始，然后逐步详细介绍各个组件。

3.1 Transformer 语言模型

给定一个标记 ID 序列，Transformer 语言模型使用输入嵌入将标记 ID 转换为密集向量，然后将嵌入后的标记通过 `num_layers` 个 Transformer 块，再应用一个学习到的线性投影（“输出嵌入”或“LM 头”）来生成预测的下一个标记的 logits。请参见图 1 的示意图。

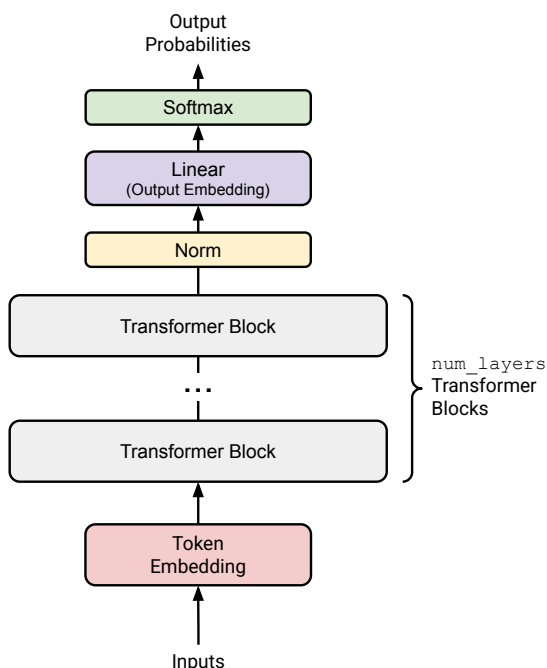


图 1：我们的 Transformer 语言模型概述。

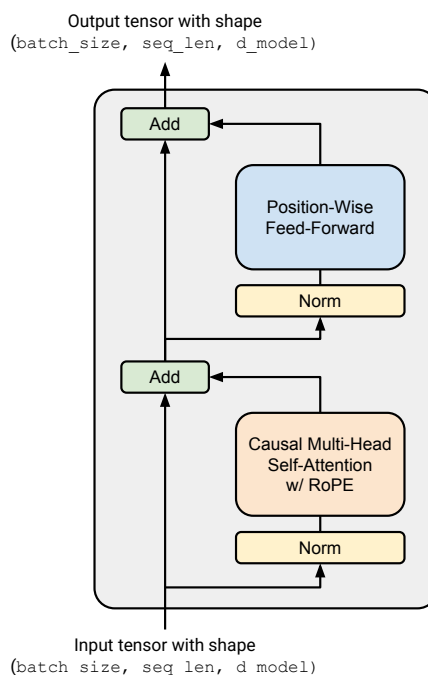


图 2：一个预归一化 Transformer 模块。

词元嵌入

在第一步中，Transformer 将（批处理的）令牌 ID 序列转换为包含令牌身份信息的向量序列（图 1 中的红色块）。

更具体地说，给定一系列的令牌 ID，Transformer 语言模型使用令牌嵌入层来生成向量序列。每个嵌入层接收形状为 $(\text{batch_size}, \text{sequence_length})$ 的整数张量，并生成形状为 $(\text{batch_size}, \text{sequence_length}, d_{\text{model}})$ 的向量序列。

预规范变压器模块

在嵌入之后，激活由几个结构完全相同的神经网络层处理。标准的仅解码器 Transformer 语言模型由 num_layers 个相同的层（通常称为 Transformer “块”）组成。每个 Transformer 块的输入形状为 $(\text{batch_size}, \text{sequence_length}, d_{\text{model}})$ ，输出形状为 $(\text{batch_size}, \text{sequence_length}, d_{\text{model}})$ 。每个块通过序列聚合信息（通过自注意力）并对其进行非线性变换

（通过前馈层）。

在经过 num_layers 个 Transformer 块之后，我们将获取最终的激活并将其转换为词汇表上的分布。

我们将实现“预归一化”Transformer 模块（详见第 3.4 节），这还需要在最后一个 Transformer 模块之后使用层归一化（详见下文），以确保其输出被适当缩放。

在这一归一化之后，我们将使用一个标准的学习线性变换将 Transformer 块的输出转换为预测的下一个 token 的 logits（参见例如 A. Radford 等人 [7] 式 2）。

3.2 备注：批处理、Einsum 和高效计算

在整个 Transformer 中，我们将对许多类似批次的输入执行相同的计算。这里有几个例子：

- 批次的元素：我们在每个批次元素上应用相同的 Transformer forward 操作。
- 序列长度：像 RMSNorm 和前馈这样的“按位置”操作在序列的每个位置上执行方式相同。
- 注意力头：在“多头”注意力操作中，注意力操作会在注意力头之间批量执行。

以一种符合人体工学的方式执行此类操作是有用的，这种方式能够充分利用 GPU，并且易于阅读和理解。许多 PyTorch 操作可以在张量的开头接收过多的“类似批次”的维度，并在这些维度上高效地重复/广播操作。

例如，假设我们正在进行按位置的批量操作。我们有一个形状为 $(\text{batch_size}, \text{sequence_length}, d_{\text{model}})$ 的“数据张量” D ，并且我们希望对形状为 $(d_{\text{model}}, d_{\text{model}})$ 的矩阵 A 进行批量向量-矩阵乘法。在这种情况下， $D @ A$ 将执行批量矩阵乘法，这是 PyTorch 中一个高效的原子，其中 $(\text{batch_size}, \text{sequence_length})$ 维度是批量处理的。

因此，假设你的函数可能会接收到额外的类似批次的维度，并将这些维度保留在 PyTorch 形状的开头是有帮助的。为了以这种方式组织张量以便批处理，它们可能需要通过多步 view、reshape 和 transpose 来调整形状。这可能有点麻烦，而且代码的可读性以及张量的形状通常会变得难以理解。

一个更符合人体工学的选择是在 `torch.einsum` 中使用 *einsum notation*, 或者更准确地说, 使用像 `einops` 或 `einx` 这样的框架无关库。两个关键操作是 `einsum`, 它可以对输入张量的任意维度进行张量收缩, 以及 `rearrange`, 它可以重新排序、连接和拆分任意维度。事实证明, 几乎所有的机器学习操作都是维度转换与张量收缩的某种组合, 偶尔会有 (通常是逐点的) 非线性函数。这意味着, 当使用 `einsum` 表示法时, 你的许多代码可以更易读且更灵活。

我们强烈建议在课堂上学习并使用 `einsum` 符号表示法。以前没有接触过 `einsum` 符号表示法的学生应该使用 `einops` (文档在这里), 已经熟悉 `einops` 的学生应该学习更通用的 `einx` (在这里)。⁴ 我们提供的环境中已经安装了这两个软件包。

在这里, 我们给出一些 `einsum` 符号的使用示例。这些是 `einops` 文档的补充, 您应首先阅读该文档。

Example (einstein_example1): Batched matrix multiplication with `einops.einsum`

```
import torch
from einops import rearrange, einsum

## Basic implementation
Y = D @ A.T
# Hard to tell the input and output shapes and what they mean.
# What shapes can D and A have, and do any of these have unexpected behavior?

## Einsum is self-documenting and robust
#           D           A           ->           Y
Y = einsum(D, A, "batch sequence d_in, d_out d_in -> batch sequence d_out")

## Or, a batched version where D can have any leading dimensions but A is constrained.
Y = einsum(D, A, "... d_in, d_out d_in -> ... d_out")
```

Example (einstein_example2): Broadcasted operations with `einops.rearrange`

We have a batch of images, and for each image we want to generate 10 dimmed versions based on some scaling factor:

```
images = torch.randn(64, 128, 128, 3) # (batch, height, width, channel)
dim_by = torch.linspace(start=0.0, end=1.0, steps=10)

## Reshape and multiply
dim_value = rearrange(dim_by, "dim_value -> 1 dim_value 1 1 1")
images_rearr = rearrange(images, "b height width channel -> b 1 height width channel")
dimmed_images = images_rearr * dim_value

## Or in one go:
dimmed_images = einsum(
    images, dim_by,
    "batch height width channel, dim_value -> batch dim_value height width channel"
)
```

⁴It's worth noting that while `einops` has a great amount of support, `einx` is not as battle-tested. You should feel free to fall back to using `einops` with some more plain PyTorch if you find any limitations or bugs in `einx`.

Example (einstein_example3): Pixel mixing with `einops.rearrange`

Suppose we have a batch of images represented as a tensor of shape (batch, height, width, channel), and we want to perform a linear transformation across all pixels of the image, but this transformation should happen independently for each channel. Our linear transformation is represented as a matrix B of shape (height * width, height * width).

```
channels_last = torch.randn(64, 32, 32, 3) # (batch, height, width, channel)
B = torch.randn(32*32, 32*32)

## Rearrange an image tensor for mixing across all pixels
channels_last_flat = channels_last.view(
    -1, channels_last.size(1) * channels_last.size(2), channels_last.size(3)
)
channels_first_flat = channels_last_flat.transpose(1, 2)
channels_first_flat_transformed = channels_first_flat @ B.T
channels_last_flat_transformed = channels_first_flat_transformed.transpose(1, 2)
channels_last_transformed = channels_last_flat_transformed.view(*channels_last.shape)
```

Instead, using `einops`:

```
height = width = 32
## Rearrange replaces clunky torch view + transpose
channels_first = rearrange(
    channels_last,
    "batch height width channel -> batch channel (height width)"
)
channels_first_transformed = einsum(
    channels_first, B,
    "batch channel pixel_in, pixel_out pixel_in -> batch channel pixel_out"
)
channels_last_transformed = rearrange(
    channels_first_transformed,
    "batch channel (height width) -> batch height width channel",
    height=height, width=width
)
```

Or, if you're feeling crazy: all in one go using `einx.dot` (`einx` equivalent of `einops.einsum`)

```
height = width = 32
channels_last_transformed = einx.dot(
    "batch row_in col_in channel, (row_out col_out) (row_in col_in)"
    "-> batch row_out col_out channel",
    channels_last, B,
    col_in=width, col_out=width
)
```

The first implementation here could be improved by placing comments before and after to indicate what the input and output shapes are, but this is clunky and susceptible to bugs. With `einsum` notation, documentation *is* implementation!

`Einsum` notation can handle arbitrary input batching dimensions, but also has the key benefit of being *self-documenting*. It's much clearer what the relevant shapes of your input and output tensors are in code

使用einsum表示法。对于其余的张量，你可以考虑使用Tensor类型提示，例如使用jaxtyping库（不特定于JAX）。

我们将在作业2中更多地讨论使用einsum符号的性能影响，但现在要知道，它们几乎总是比替代方案更好！

3.2.1 数学符号和内存排序

许多机器学习论文在其符号中使用行向量，这导致的表示与 NumPy 和 PyTorch 默认使用的行优先内存顺序非常匹配。使用行向量时，线性变换看起来像

$$y = xW^T, \quad (1)$$

对于行主序的 $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ 和行向量的 $x \in \mathbb{R}^{1 \times d_{\text{in}}}$ 。注意，这使我们能够通过增加 x 的最外层维度来批处理输入，这意味着我们可以用向量输入 x 替代矩阵输入 $X \in \mathbb{R}^{\text{batch} \times d_{\text{in}}}$ 。

在线性代数中，通常更常用列向量，其中线性变换看起来像

$$y = Wx, \quad (2)$$

给定一个行优先的 $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ 和列向量 $x \in \mathbb{R}^{d_{\text{in}}}$ 。要在这种设置下批处理输入，批处理维度 x 必须放在最后，因此 x 需要被替换为一个矩阵 $X \in \mathbb{R}^{d_{\text{in}} \times \text{batch}}$ 。

在本次作业中，我们将主要使用列向量进行数学符号表示，因为数学通常采用这种表示法。你应该记住，如果你想使用普通的矩阵乘法表示法，你将不得不像在方程式 1 的行向量惯例中那样应用转置矩阵，因为 PyTorch 使用行主序存储。如果你在进行线性代数运算时使用 einsum，只要正确标记你的轴，这应该不会成为问题。顺便提一下，值得注意的是，其他语言/线性代数包如 Matlab、Julia 和 Fortran 都使用列主序存储，也就是 *batching dimensions come last*，但 Python 及其相关包采用了 C 标准的行主序存储。

3.3 基本账户 构建模块：线性与嵌入模块

度尔斯

3.3.1 参数初始化

有效训练神经网络通常需要对模型参数进行仔细初始化——不良的初始化可能导致梯度消失或爆炸等不良行为。Pre-norm 变换器对初始化异常鲁棒，但它们仍可能对训练速度和收敛性产生显著影响。由于本次作业已经很长，我们将在作业 3 中详细介绍这些内容，这里我们提供一些近似的初始化方法，应该适用于大多数情况。

目前，使用：

- 线性权重： $\mathcal{N}(\mu = 0, \sigma^2 = \frac{2}{d_{\text{in}} + d_{\text{out}}})$ 截断于 $[-3\sigma, 3\sigma]$ 。
- 嵌入： $\mathcal{N}(\mu = 0, \sigma^2 = 1)$ 在 $[-3, 3]$ 截断
- RMSNorm: 1

你应该使用 `torch.nn.init.trunc_normal_` 来初始化截断正态权重。

3.3.2 线性模块

线性层是变压器和神经网络的一种基本构建模块。首先，您将实现自己继承自 `torch.nn.Module` 的 `Linear` 类，并执行线性变换：

$$y = Wx. \quad (3)$$

注意 th 我们不包括偏置项，遵循大多数现代做法

大型语言模型

问题 (**linear**): 实现线性模块 (1 分)

交付成果：实现一个继承自 `torch.nn.Module` 的 `Linear` 类，并执行线性变换。您的实现应遵循 PyTorch 内置 `nn.Linear` 模块的接口，但不应包含 `bias` 参数或参数。我们推荐以下接口：

`def __init__(self, in_features, out_features, device=None, dtype=None)` 构建一个线性变换模块。此函数应接受以下参数：

`in_features: int` 输入的最终维度

`out_features: int` 输出的最终维度

`device: torch.device | None = None` 用于存储参数的设备

`dtype: torch.dtype | None = None` 参数的数据类型

`def forward(self, x: torch.Tensor) -> torch.Tensor` 将线性变换应用到输入上。

确保：

- 子类 `nn.Module`
- 调用父类构造函数
- 将你的参数构建并存储为 W （而不是 W^T ），将其放入 `nn.Parameter` 中
- 当然，不要使用 `nn.Linear` 或 `nn.functional.linear`

对于初始化，使用上面的设置以及 `torch.nn.init.trunc_normal_` 来初始化权重。

要测试你的 `Linear` 模块，请在 `[adapters.run_linear]` 实现测试适配器。该适配器应将给定的权重加载到你的 `Linear` 模块中。你可以使用 `Module.load_state_dict` 来完成此目的。然后，运行 `uv run pytest -k test_linear`。

3.3.3 嵌入模块

如上所述，Transformer 的第一层是一个嵌入层，将整数 token ID 映射到维度为 `d_model` 的向量空间中。我们将实现一个自定义的 `Embedding` 类，该类继承自 `torch.nn.Module`（因此你不应该使用 `nn.Embedding`）。`forward` 方法应通过使用形状为 `(batch_size, sequence_length)` 的 token ID `torch.LongTensor` 对形状为 `(vocab_size, d_model)` 的嵌入矩阵进行索引，为每个 token ID 选择嵌入向量。

问题 (**embedding**): 实现嵌入模块 (1 分)

交付成果：实现继承自 `torch.nn.Module` 的 `Embedding` 类，并执行嵌入查找。您的实现应遵循 PyTorch 内置 `nn.Embedding` 模块的接口。我们建议以下接口：

`def __init__(self, num_embeddings, embedding_dim, device=None, dtype=None)` 构建一个嵌入模块。此函数应接受以下参数：

num_embeddings: int 词汇量的大小

embedding_dim: int 嵌入向量的维度，即 d_{model}

device: torch.device | None = None 用于存储参数的设备

dtype: torch.dtype | None = None 参数的数据类型

`def forward(self, token_ids: torch.Tensor) -> torch.Tensor` 查找给定标记 ID 的嵌入向量。

确保：

- 子类 `nn.Module`
- 调用父类构造函数
- 将你的嵌入矩阵初始化为 `nn.Parameter`
- 将嵌入矩阵存储为 `d_model` 作为最终维度
- 当然，不要使用 `nn.Embedding` 或 `nn.functional.embedding`

再次，使用上面的设置进行初始化，并使用 `torch.nn.init.trunc_normal_` 来初始化权重。

要测试你的实现，请在 `[adapters.run_embedding]` 实现测试适配器。然后，运行 `uv run pytest -k test_embedding`。

3.4 预归一化变压器模块

每个 Transformer 块有两个子层：多头自注意力机制和逐位置前馈网络（[A. Vaswani 等, 2017], 第 3.1 节）。

在最初的 Transformer 论文中，该模型在两个子层的每一层周围都使用了残差连接，然后进行层归一化。这种架构通常被称为“后归一化（post-norm）”Transformer，因为层归一化是应用在子层输出上的。然而，大量研究发现，将层归一化从每个子层的输出移动到每个子层的输入（在最终 Transformer 块之后再加一个层归一化）可以提高 Transformer 的训练稳定性 [T. Q. Nguyen 等人, 2019; R. Xiong 等人, 2020] —— 参见图 2，该图直观展示了这种“前归一化（pre-norm）”Transformer 块。每个 Transformer 块子层的输出随后通过残差连接加到子层输入上（A. Vaswani 等人 [8], 第 5.4 节）。前归一化的直觉是存在一个干净的“残差流”，没有任何归一化，从输入嵌入一直流向 Transformer 的最终输出，这被认为

3.4.1 均方根层归一化

A. Vaswani 等人 [8] 的原始 Transformer 实现使用层归一化 [J. L. Ba 等人, 2016] 来归一化激活值。遵循 H. T. Ouyron 等人 [12] 的做法，我们将使用平方根均方层归一化（RMSNorm；B. Zhang 等人 [13], 公式 4）进行层归一化。给定一个激活向量 $a \in \mathbb{R}^{d_{\text{model}}}$ ，RMSNorm 将按如下方式重新缩放每个激活 a_i ：

$$\text{RMSNorm}(a_i) = \frac{a_i}{\text{RMS}(a)} g_i, \quad (4)$$

其中 $\text{RMS}(a) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{i=1}^{d_{\text{model}}} a_i^2 + \varepsilon}$ 。这里， g_i 是一个可学习的“增益”参数（总共有 d_{model} 个这样的参数），而 ε 是一个通常固定为 $1e-5$ 的超参数。

你应该将输入提升为 `torch.float32` 以防在平方输入时发生溢出。总体来说，你的 `forward` 方法应如下所示：

```
in_dtype = x.dtype
x = x.to(torch.float32)

# Your code here performing RMSNorm
...
result = ...

# Return the result in the original dtype
return result.to(in_dtype)
```

问题 (rms norm): 均方根层归一化 (1 p 关节)

交付成果：将 `RMSNorm` 实现为 `torch.nn.Module`。我们推荐以下接口：

def __init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None) 构建 `RMSNorm` 模块。此函数应接受以下参数：

d_model: int 模型的隐藏维度

eps: float = 1e-5 用于数值稳定性的 Epsilon 值

device: torch.device | None = None 存储参数的设备

dtype: torch.dtype | None = None 参数的数据类型

def forward(self, x: torch.Tensor) -> torch.Tensor 处理形状为 `(batch_size, sequence_length, d_model)` 的输入张量，并返回相同形状的张量。

注意：如上所述，在执行归一化之前，记得将输入上转换为 `torch.float32`（之后再转换回原始数据类型）。

要测试你的实现，请在 `[adapters.run_rmsnorm]` 实现测试适配器。然后，运行 `uv run pytest -k test_rmsnorm`。

3.4.2 逐位置前馈网络

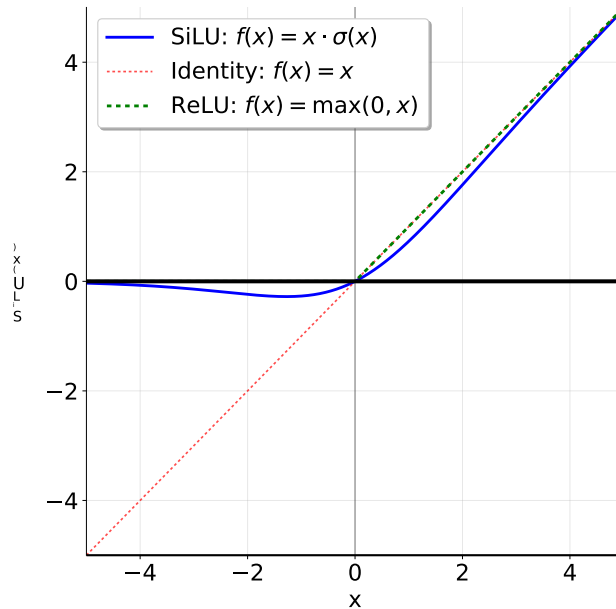


图 3: 比较SiLU（也称Swish）和ReLU激活函数。

在原始的 Transformer 论文中（A. Vaswani 等人 [8] 的第 3.3 节），Transformer 前馈网络由两个线性变换组成，它们之间有一个 ReLU 激活（ $\text{ReLU}(x) = \max(0, x)$ ）。在该原始架构中，内部前馈层的维度通常是输入维度的 4 倍。

然而，与最初的设计相比，现代语言模型倾向于包含两个主要变化：它们使用另一种激活函数，并采用门控机制。具体来说，我们将实现大语言模型（如 Llama 3 [A. Grattafiori et al., 2024] 和 Qwen 2.5 [A. Yang et al., 2024]）采用的“SwiGLU”激活函数，它将 SiLU（通常称为 Swish）激活函数与一种称为门控线性单元（GLU）的门控机制结合。同时，我们也将省略线性层中有时使用的偏置项，这与大多数现代大语言模型保持一致，自 PaLM [A. Chowdhery et al., 2022] 和 LLaMA [H. Touvron et al., 2023] 以来如此。

SiLU 或 Swish 激活函数 [D. Hendrycks 等, 2016; S. Elfwing 等, 2017] 定义如下：

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \quad (5)$$

如图 3 所示，SiLU 激活函数类似于 ReLU 激活函数，但在零点处是平滑的。

门控线性单元（GLUs）最初由 Y. N. Dauphin 等人 [19] 定义为经过 sigmoid 函数的线性变换与另一线性变换的逐元素乘积：

$$\text{GLU}(x, W_1, W_2) = \sigma(W_1 x) \odot W_2 x, \quad (6)$$

其中 $\odot$ 表示逐元素相乘。建议使用门控线性单元，以“通过为梯度提供线性路径的同时保留非线性能力，从而减少深层架构的梯度消失问题。”

将 SiLU/Swish 和 GLU 组合在一起，我们得到 SwiGLU，我们将在前馈网络中使用它：

$$\text{FFN}(x) = \text{SwiGLU}(x, W_1, W_2, W_3) = W_2(\text{SiLU}(W_1 x) \odot W_3 x), \quad (7)$$

其中 $x \in \mathbb{R}^{d_{\text{model}}}$ 、 $W_1, W_3 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ 、 $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ ，以及规范上， $d_{\text{ff}} = \frac{8}{3} d_{\text{model}}$ 。对于具体实现，将其四舍五入到接近的64的倍数以提高硬件效率是可以的。

[N. Shazeer \[20\]](#) 最早提出将 SiLU/Swish 激活函数与 GLU 结合，并进行了实验，显示 SwiGLU 在语言建模任务上优于 ReLU 和未加门控的 SiLU 基线。稍后在作业中，你将比较 SwiGLU 和 SiLU。虽然我们提到了关于这些组件的启发性论点（相关论文提供了更多支持证据），但保持实证视角是有益的：Shazeer 论文中有一句现在已很有名的引用是

“我们没有解释为什么这些架构似乎有效；我们将它们的成功，和其他一切一样，归因于神的仁慈。”

问题 (**positionwise_feedforward**): 实现逐位置前馈网络 (2 分)

可交付成果：实现 SwiGLU 前馈网络，由 SiLU 激活函数和 GLU 组成。

注意：在这个特殊情况下，你可以在实现中自由使用 `torch.sigmoid` 以保证数值稳定性。

在你的实现中，你应该将 d_{ff} 设置为大约 $\frac{8}{3} \times d_{\text{model}}$ ，同时确保内部前馈层的维度是 64 的倍数，以充分利用你的硬件。要使用我们提供的测试来测试你的实现，你需要在 `[adapters.run_swiglu]` 实现测试适配器。然后，运行 `uv run pytest -k test_swiglu` 来测试你的实现。

3.4.3 相对位置嵌入

为了将位置信息注入模型，我们将实现旋转位置嵌入（Rotary Position Embeddings）[J. Su 等, 2021]，通常称为 **RoPE**。对于给定的查询标记 $q^{(i)} = W_q x^{(i)} \in \mathbb{R}^d$ 在标记位置 i ，我们将应用一个成对的旋转矩阵 R^i ，得到 $q'^{(i)} = R^i q^{(i)} = R^i W_q x^{(i)}$ 。这里， R^i 将以角度 $\theta_{i,k} = \frac{i}{\Theta(2k-2)/d}$ 将嵌入元素对 $q_{2k-1:2k}^{(i)}$ 作为二维向量旋转，用于 $k \in \{1, \dots, d/2\}$ 和某个常数 Θ 。因此，我们可以将 R^i 视为一个大小为 $d \times d$ 的块对角矩阵，其块为 R_k^i 对 $k \in \{1, \dots, \frac{d}{2}\}$ ，其中

$$R_k^i = \begin{pmatrix} \cos(\theta_{i,k}) & -\sin(\theta_{i,k}) \\ \sin(\theta_{i,k}) & \cos(\theta_{i,k}) \end{pmatrix} \quad (8)$$

因此我们得到完整的旋转矩阵

$$R^i = \begin{pmatrix} R_1^i & 0 & 0 & \cdots & 0 \\ 0 & R_2^i & 0 & \cdots & 0 \\ 0 & 0 & R_3^i & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & R_{d/2}^i \end{pmatrix}, \quad (9)$$

0 表示 2×2 个零矩阵。虽然可以构建完整的 $d \times d$ 矩阵，但一个好的解决方案应该利用该矩阵的特性来更高效地实现转换。由于我们只关心给定序列内标记的相对旋转，我们可以在层之间以及不同批次中重用为 $\cos(\theta_{i,k})$ 和 $\sin(\theta_{i,k})$ 计算的值。如果你想进行优化，你可以使用一个由所有层引用的单个 RoPE 模块，并且它可以有一个在初始化时使用 `self.register_buffer(persistent=False)` 创建的 `sin` 和 `cos` 值的二维预计算缓存，而不是 `nn.Parameter`（因为我们不想学习这些固定的余弦和正弦值）。随后，对 $k^{(j)}$ 做的旋转过程与我们对 $q^{(i)}$ 所做的完全相同，旋转角度为对应的 R^j 。请注意，这一层没有可学习的参数。

问题（rope）： 实现 RoPE（2 分）

交付成果：实现一个类 `RotaryPositionalEmbedding`，将 RoPE 应用于输入张量。

推荐使用以下接口：

def __init__(self, theta: float, d_k: int, max_seq_len: int, device=None) 构建 RoPE 模块并在需要时创建缓冲区。

theta: float Θ RoPE 的值

d_k: int 查询向量和键向量的维度

max_seq_len: int 将被输入的最大序列长度

device: torch.device | None = None 用于存储缓冲区的设备

def forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor 处理形状为 $(\dots, \text{seq_len}, d_k)$ 的输入张量，并返回相同形状的张量。请注意，你应该能够容忍具有任意数量批次维度的 x 。你应该假设 `token` 位置是形状为 $(\dots, \text{seq_len})$ 的张量，指定序列维度上 x 的 `token` 位置。

您应该使用 `token` 位置沿序列维度切片您（可能已预计算的）`cos` 和 `sin` 张量。

要测试你的实现，请完成 `[adapters.run_rope]` 并确保它通过 `uv run pytest -k test_rope`。

3.4.4 缩放点积注意力

我们现在将实现 A. Vaswani 等人 [8]（第 3.2.1 节）中描述的缩放点积注意力。

作为初步步骤，Attention 操作的定义将使用 softmax，这是一种将未归一化的分数向量转换为归一化分布的操作：

$$\text{softmax}(v)_i = \frac{\exp(v_i)}{\sum_{j=1}^n \exp(v_j)}. \quad (10)$$

请注意，对于较大的值， $\exp(v_i)$ 可能会变为 `inf`（然后， $\frac{\inf}{\inf} = \text{NaN}$ ）。我们可以注意到 `softmax` 操作在将任意常数 c 加到所有输入上时是不变的，从而避免这种情况。我们可以利用这个性质来提高数值稳定性——通常，我们会从 v 的所有元素中减去 v 的最大值，使得新的最大值为 0。现在，您将实现 `softmax`，使用这个技巧来提高数值稳定性。

问题 (**softmax**): 实现 `softmax` (1 分)

交付成果：编写一个函数，对张量执行 `softmax` 操作。你的函数应接受两个参数：一个张量和一个 *dimension* i ，并对输入张量的第 i 维应用 `softmax`。输出张量应与输入张量形状相同，但其第 i 维现在将具有归一化的概率分布。使用从第 i 维的所有元素中减去最大值的技巧来避免数值稳定性问题。

要测试你的实现，请完成 `[adapters.run_softmax]` 并确保它通过 `uv run pytest -k test_softmax_matches_pytorch`。

我们现在可以用数学方式定义注意力操作如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (11)$$

其中 $Q \in \mathbb{R}^{n \times d_k}$ 、 $K \in \mathbb{R}^{m \times d_k}$ 和 $V \in \mathbb{R}^{m \times d_v}$ 。这里， Q 、 K 和 V 都是这个操作的输入——注意这些不是可学习的参数。

掩码：有时对注意力操作的输出进行掩码处理是方便的。掩码的形状应为 $M \in \{\text{True}, \text{False}\}^{n \times m}$ ，该布尔矩阵的每一行 i 表示查询 i 应该关注哪些键。按照惯例（并且有些令人困惑），在位置 (i, j) 的值为 `True` 表示查询 i *does* 不应该关注键 j ，而值为 `False` 表示查询 *does not* 应该关注该键。换句话说，“信息流”在 (i, j) 对应于值 `True`。例如，考虑一个掩码矩阵，其条目为 `[[True, True, False]]`。该单个查询向量只关注前两个键。

在计算上，使用掩码比在子序列上计算注意力要高效得多，我们可以通过取预 `softmax` 值 $(QK^T / \sqrt{d_k})$ 并将 $-\infty$ 添加到掩码矩阵中为 `False` 的任何条目来实现这一点。

问题 (**scaled_dot_product_attention**): 实现缩放点积注意力 (5 分)

交付成果：实现缩放点积注意力函数。你的实现应能处理形状为 `(batch_size, ..., seq_len, d_k)` 的键和值为形状 `(batch_size, ..., seq_len, d_v)` 的查询，其中 `...` 表示任何其他批处理类维度（如果提供）。实现应返回形状为 `(batch_size, ..., seq_len, d_v)` 的输出。有关批处理类维度的讨论，请参见第 3.2 节。

您的实现还应支持用户提供的可选布尔掩码，形状为 `(seq_len, seq_len)`。掩码值为 `True` 的位置的注意力概率应总和为 1，而掩码值为 `False` 的位置的注意力概率应为零。

要使用我们提供的测试来测试您的实现，您需要在 `[adapters.run_scaled_dot_product_attention]` 实现测试适配器。运行 `uv run pytest -k test_scaled_dot_product_attention` 测试您的实现对于三阶输入张量的情况，而运行 `uv run pytest -k test_4d_scaled_dot_product_attention` 测试您的实现对于四阶输入张量的情况。

3.4.5 因果多头自注意力

我们将实现多头自注意力，如 A. Vaswani 等人 [8] 第 3.2.2 节中所述。回顾一下，从数学上讲，应用多头注意力的操作定义如下：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \quad (12)$$

$$\text{for head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (13)$$

其中 Q_i 、 K_i 、 V_i 分别是第 $i \in \{1, \dots, h\}$ 个切片，大小为 d_k 或 d_v ，对应于 Q, K 和 V 。其中 `Attention` 是第 3.4.4 节中定义的缩放点积注意力操作。由此我们可以形成多头 *self* 注意力操作：

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x) \quad (14)$$

这里，可学习的参数是 $W_Q \in \mathbb{R}^{hd_k \times d_{\text{model}}}$ 、 $W_K \in \mathbb{R}^{hd_k \times d_{\text{model}}}$ 、 $W_V \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ 和 $W_O \in \mathbb{R}^{d_{\text{model}} \times hd_v}$ 。由于在多头注意力操作中 Q 、 K 和 V 被切分，我们可以将 W_Q 、 W_K 和 W_V 看作沿输出维度为每个头分开。当你完成这一操作时，你应该在总共三次矩阵乘法中计算键、值和查询的投影。⁵

因果掩码

您的实现应当防止模型关注序列中的未来标记。换句话说，如果模型给定一个标记序列 $t_1, \dots, t_n$ ，并且我们想要计算前缀 $t_1, \dots, t_i$ 的下一个词预测（其中 $i < n$ ），模型不应能够访问（关注）位置 $t_{i+1}, \dots, t_n$ 的标记表示，因为在推理生成文本时它无法访问这些标记（而这些未来标记会泄露真实下一个词的身份，使语言建模预训练目标变得简单）。对于输入标记序列 $t_1, \dots, t_n$ ，我们可以通过对序列中 n 个唯一前缀运行多头自注意力 n 次来天真地阻止访问未来标记。相反，我们将使用因果注意掩码，这允许标记 i 关注序列中的所有位置 $j \leq i$ 。您可以使用 `torch.triu` 或广播索引比较来构建此掩码，并且您应该利用

应用 RoPE

RoPE 应该应用于查询向量和键向量，但不应用于值向量。此外，头部维度应作为批处理维度来处理，因为在多头注意力中，每个头的注意力是独立应用的。这意味着对于每个头，应该将完全相同的 RoPE 旋转应用于查询向量和键向量。

问题 (`multihead_self_attention`): 实现因果多头自注意力 (5 分)

⁵As a stretch goal, try combining the key, query, and value projections into a single weight matrix so you only need a single matrix multiply.

交付成果：将因果多头自注意力实现为 `torch.nn.Module`。您的实现应至少接受以下参数：

d_model: `int`Transformer 块输入的维度。

num_heads: `int`在多头自注意力中使用的头数。

根据 A. Vaswani 等人 [8]，设置 $d_k = d_v = \frac{d_{\text{model}}}{h}$ 。要将你的实现与我们提供的测试进行对比，请在 `[adapters.run_multihead_self_attention]` 实现测试适配器。然后，运行 `uv run pytest -k test_multihead_self_attention` 来测试你的实现。

3.5 完整的 Transformer 语言模型

让我们先组装 Transformer 块（参考图 2 会很有帮助）。一个 Transformer 块包含两个“子层”，一个用于多头自注意力，另一个用于 SwiGLU 前馈网络。在每个子层中，我们首先执行 RMSNorm，然后执行主要操作（多头自注意力/前馈网络），最后加入残差连接。

具体来说，Transformer 块的前半部分（第一个“子层”）应实现以下一组更新，以便从输入 x 生成输出 y ，

$$y = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)). \quad (15)$$

问题 (**transformer_block**): 实现 Transformer 模块 (3 分)

按照第 3.4 节的描述并如图 2 所示，实现预归一化 Transformer 块。你的 Transformer 块应接受（至少）以下参数。

d_model: `int`Transformer 块输入的维度。

num_heads: `int`在多头自注意力中使用的头数。

d_ff: `int`逐位置前馈内部层的维度。

要测试你的实现，请实现适配器 `[adapters.run_transformer_block]`。然后运行 `uv run pytest -k test_transformer_block` 来测试你的实现。

交付物：通过提供的测试的变压器模块代码。

现在我们将模块组合在一起，按照图 1 中的高层次示意图进行操作。按照我们在第 3.1.0.1 节中对嵌入的描述，将其输入到 `num_layers` Transformer 模块中，然后将其传入最终的层归一化和语言模型头，以获得词汇表上的未归一化分布（logits）。

问题 (**transformer_lm**): 实现 Transformer 语言模型 (3 分)

是时候把所有东西组合在一起了！按照第 3.1 节的描述并如图 1 所示，实现 Transformer 语言模型。至少，你的实现应接受上述所有 Transformer 模块的构建参数，以及以下附加参数：

vocab_size: `int`词汇量的大小，是确定词元嵌入矩阵维度所必需的。

context_length: int 确定 RoPE sin 和 cos 缓冲区维度所需的最大上下文长度。

num_layers: int 要使用的 Transformer 块的数量。

要将你的实现与我们提供的测试进行比较，你首先需要在 `[adapters.run_transformer_lm]` 实现测试适配器。然后，运行 `uv run pytest -k test_transformer_lm` 来测试你的实现。

交付物：一个能够通过上述测试的 Transformer 语言模型模块。

资源会计

能够理解 Transformer 的各个部分如何消耗计算资源和内存是很有用的。我们将逐步进行一些基本的 “FLOPs 计算”。在 Transformer 中，大多数的 FLOPs 是矩阵乘法，因此我们的核心方法很简单：

1. 写下 Transformer 前向传播中的所有矩阵乘法。
2. 将每个矩阵乘法转换为所需的浮点运算数 (FLOPs)。

对于第二步，以下事实将会有用：

规则：给定 $A \in \mathbb{R}^{m \times n}$ 和 $B \in \mathbb{R}^{n \times p}$ ，矩阵-矩阵乘积 AB 需要 $2mnp$ 次浮点运算。

为了看出这一点，请注意 $(AB)[i, j] = A[i, :] \cdot B[:, j]$ ，并且这个点积需要 n 次加法和 n 次乘法 ($2n$ 次浮点运算)。然后，由于矩阵乘法 AB 有 $m \times p$ 个元素，总的浮点运算次数是 $(2n)(mp) = 2mnp$ 。

现在，在你做下一个问题之前，浏览你的 Transformer 块和 Transformer 语言模型的每个组件，列出所有矩阵乘法及其相关的 FLOPs 成本，可能会很有帮助。

问题 (**transformer_accounting**): 变压器语言模型资源核算 (5 分)

- (a) 考虑一个使用我们分配架构的 GPT-2 XL 大小模型，其具有以下配置：

vocab_size: 50,257
context_length: 1,024
num_layers: 48
d_model: 1,600
num_heads: 25
d_ff: 4,288 (最接近 $\frac{8}{3} \times 1,600$ 的 64 的倍数)

假设我们使用这个配置构建了我们的模型。我们的模型将有多少可训练参数？假设每个参数都使用单精度浮点数表示，仅加载这个模型需要多少内存？

交付成果：一到两句的回应。

- (b) 确定完成我们的 GPT-2 XL 形模型的前向传播所需的矩阵乘法。这些矩阵乘法总共需要多少 FLOPs？假设我们的输入序列有 `context_length` 个标记。

交付物：一份矩阵乘法清单（含描述）以及所需的总浮点运算次数。

(c) 根据你上面的分析，模型的哪些部分需要最多的浮点运算(FLOPs)?

交付成果：一到两句的回应。

(d) 使用 GPT-2 small（12 层，768 d_model，12 个头）、GPT-2 medium（24 层，1024 d_model，16 个头）和 GPT-2 large（36 层，1280 d_model，20 个头）重复你的分析。随着模型规模的增加，Transformer 语言模型的哪些部分在总 FLOPs 中所占的比例增加或减少？

交付物：对于每个模型，提供模型组件的细分以及其相关的 FLOPs（占前向传播所需总 FLOPs 的比例）。此外，提供一至两句描述说明模型大小的变化如何改变每个组件的 FLOPs 比例。

(e) 使用 GPT-2 XL 并将上下文长度增加到 16,384。一次前向传播的总浮点运算量（FLOPs）会如何变化？模型组件的 FLOPs 相对贡献会如何变化？

交付成果：一到两句的回应。

4 训练 Transformer 语言模型

我们现在已经有了预处理数据（通过分词器）和模型（Transformer）的步骤。剩下的就是构建所有支持训练的代码。这包括以下内容：

- 损失：我们需要定义损失函数（交叉熵）。
- 优化器：我们需要定义优化器来最小化这个损失（AdamW）。
- 训练循环：我们需要所有支持性基础设施来加载数据、保存检查点和管理训练。

4.1 交叉熵损失

回想一下，Transformer 语言模型为每个长度为 $m + 1$ 且为 $i = 1, \dots, m$ 的序列 x 定义了一个分布 $p_\theta(x_{i+1} | x_{1:i})$ 。给定一个由长度为 $m + 1$ 的序列组成的训练集 D ，我们定义标准的交叉熵（负对数似然）损失函数：

$$\ell(\theta; D) = \frac{1}{|D|} \sum_{x \in D} \sum_{i=1}^m -\log p_\theta(x_{i+1} | x_{1:i}). \quad (16)$$

(请注意，在 Transformer 中一次前向传递会产生 $p_\theta(x_{i+1} | x_{1:i})$ 用于 $\text{all } i = 1, \dots, m$.)

特别是，Transformer 为每个位置 i 计算 logits $o_i \in \mathbb{R}^{\text{vocab_size}}$ ，其结果为：⁶

$$p(x_{i+1} | x_{1:i}) = \text{softmax}(o_i)[x_{i+1}] = \frac{\exp(o_i[x_{i+1}])}{\sum_{a=1}^{\text{vocab_size}} \exp(o_i[a])}. \quad (17)$$

交叉熵损失通常是相对于对数几率向量 $o_i \in \mathbb{R}^{\text{vocab_size}}$ 和目标 x_{i+1} 来定义的。⁷

⁶Note that $o_i[k]$ refers to value at index k of the vector o_i .

⁷This corresponds to the cross-entropy between the Dirac delta distribution over x_{i+1} and the predicted $\text{softmax}(o_i)$ distribution.

实现交叉熵损失需要注意一些数值问题，就像在 softmax 的情况下那样。

问题 (**cross_entropy**): 实现交叉熵 (1 分)

可交付成果: 编写一个函数来计算交叉熵损失, 该函数接收预测的 logits (o_i) 和目标 (x_{i+1}), 并计算交叉熵 $\ell_i = -\log \text{softmax}(o_i)[x_{i+1}]$ 。你的函数应该处理以下情况:

- 为了数值稳定性, 减去最大的元素。
- 尽可能取消 log 和 exp。
- 处理任何额外的批量维度, 并返回跨批次的 *average*。与第 3.2 节一样, 我们假设批量类维度总是位于词汇量维度之前。

实现 [**adapters.run_cross_entropy**], 然后运行 `uv run pytest -k test_cross_entropy` 来测试你的实现。

困惑

交叉熵足以用于训练, 但在评估模型时, 我们也希望报告困惑度。对于长度为 m 的序列, 我们遭受了交叉熵损失 $\ell_1, \dots, \ell_m$:

$$\text{perplexity} = \exp\left(\frac{1}{m} \sum_{i=1}^m \ell_i\right). \quad (18)$$

4.2 SGD 优化器

既然我们有了损失函数, 我们将开始探索优化器。最简单的基于梯度的优化器是随机梯度下降 (SGD)。我们从随机初始化的参数 θ_0 开始。然后在每一步 $t = 0, \dots, T - 1$ 中, 我们执行以下更新:

$$\theta_{t+1} \leftarrow \theta_t - \alpha_t \nabla L(\theta_t; B_t), \quad (19)$$

其中 B_t 是从数据集 D 中抽取的随机数据批次, 而 *learning rate* α_t 和 *batch size* $|B_t|$ 是超参数。

4.2.1 在 PyTorch 中实现 SGD

为了实现我们的优化器, 我们将继承 PyTorch `torch.optim.Optimizer` 类。一个 `Optimizer` 子类必须实现两个方法:

def __init__(self, params, ...) 应该初始化你的优化器。在这里, `params` 将是一组待优化的参数 (或参数组, 如果用户希望为模型的不同部分使用不同的超参数, 例如学习率)。确保将 `params` 传递给基类的 `__init__` 方法, 这将存储这些参数以供 `step` 使用。你可以根据优化器传递额外的参数 (例如学习率是一个常见的参数), 并将它们作为字典传递给基类的构造函数, 其中键是你为这些参数选择的名称 (字符串)。

def step(self) 应该对参数进行一次更新。在训练循环中, 这将在反向传播之后被调用, 因此您可以访问最后一批的梯度。此方法应遍历每个参数张量 `p` 并修改它们 *in place*, 即设置 `p.data`, 它包含

与该参数相关的张量，基于梯度 `p.grad`（如果存在），该张量表示损失相对于该参数的梯度

PyTorch 优化器 API 有一些细微之处，所以用一个例子来解释会更容易。为了让我们的例子更丰富，我们将实现一种稍微变化的 SGD，其中学习率会在训练过程中衰减，从初始学习率 α 开始，并随着时间的推移采取逐渐变小的步骤：

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{t+1}} \nabla L(\theta_t; B_t) \quad (20)$$

让我们来看看这个版本的 SGD 将如何作为 PyTorch Optimizer 实现：

```
from collections.abc import Callable, Iterable
from typing import Optional
import torch
import math

class SGD(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3):
        if lr < 0:
            raise ValueError(f"Invalid learning rate: {lr}")
        defaults = {"lr": lr}
        super().__init__(params, defaults)

    def step(self, closure: Optional[Callable] = None):
        loss = None if closure is None else closure()
        for group in self.param_groups:
            lr = group["lr"] # Get the learning rate.
            for p in group["params"]:
                if p.grad is None:
                    continue

                state = self.state[p] # Get state associated with p.
                t = state.get("t", 0) # Get iteration number from the state, or 0.
                grad = p.grad.data # Get the gradient of loss with respect to p.
                p.data -= lr / math.sqrt(t + 1) * grad # Update weight tensor in-place.
                state["t"] = t + 1 # Increment iteration number.

        return loss
```

在 `__init__` 中，我们将参数以及默认的超参数传递给基类构造函数（参数可能以组的形式传入，每组具有不同的超参数）。如果参数只是一个 `torch.nn.Parameter` 对象的集合，基类构造函数将创建一个单独的组并分配默认的超参数。然后，在 `step` 中，我们遍历每个参数组，然后遍历该组中的每个参数，并应用方程 20。在这里，我们将迭代次数保持为与每个参数关联的状态：我们首先读取该值，在梯度更新中使用它，然后更新它。

API 指定用户可能传入一个可调用的 `closure` 来在优化器步骤之前重新计算损失。对于我们将使用的优化器，这个功能不是必须的，但我们添加它是为了符合 API。

要看到这个工作的效果，我们可以使用以下 *training loop* 的最小示例：

```
weights = torch.nn.Parameter(5 * torch.randn((10, 10)))
opt = SGD([weights], lr=1)

for t in range(100):
    opt.zero_grad() # Reset the gradients for all learnable parameters.
    loss = (weights**2).mean() # Compute a scalar loss value.
    print(loss.cpu().item())
```

```
loss.backward() # Run backward pass, which computes gradients.
opt.step() # Run optimizer step.
```

这是训练循环的典型结构：在每次迭代中，我们将计算损失并执行优化器的一步。在训练语言模型时，我们的可学习参数将来自模型（在 PyTorch 中，`m.parameters()` 给我们提供了这个集合）。损失将基于采样的数据批次计算，但训练循环的基本结构将保持不变。

问题 (**learning_rate_tuning**): 调整学习率 (1分)

正如我们将看到的，影响训练最重要的超参数之一是学习率。让我们在我们的玩具示例中实际看看。用三个其他学习率值 `1e1`、`1e2` 和 `1e3` 运行上面的 SGD 示例，仅进行 10 次训练迭代。这些学习率的损失情况如何？它是下降得更快、更慢，还是发散（即在训练过程中增加）？

可交付成果：用一到两句话描述你观察到的行为。

4.3 AdamW

现代语言模型通常使用更复杂的优化器进行训练，而不是使用 SGD。最近使用的大多数优化器都是 Adam 优化器的衍生版本 [D. P. Kingma 等, 2015]。我们将使用 AdamW [I. Loshchilov 等, 2019]，它在最近的研究中被广泛使用。AdamW 对 Adam 提出了一个改进，通过添加 *weight decay* 提高正则化（在每次迭代中，我们将参数拉向 0），并且这种方式与梯度更新是解耦的。我们将按照 I. Loshchilov 等 [23] 的算法 2 中描述的方法实现 AdamW。

AdamW 是 *stateful*：对于每个参数，它会跟踪该参数的一阶矩和二阶矩的运行估计。因此，AdamW 使用额外的内存以换取更好的稳定性和收敛性。除了学习率 α 外，AdamW 还有一对超参数 (β_1, β_2) 用于控制矩估计的更新，以及一个权重衰减率 λ 。典型应用通常将 (β_1, β_2) 设置为 $(0.9, 0.999)$ ，但像 LLaMA [H. Touvron 等, 2023] 和 GPT-3 [T. B. Brown 等, 2020] 这样的大型语言模型通常使用 $(0.9, 0.95)$ 进行训练。该算法可以写成如下形式，其中 ϵ 是一个小值（例如 10^{-8} ），用于提高数值稳定性，以防 v 中出现非常小的值：

Algorithm 1: AdamW Optimizer

```
1 init( $\theta$ )  ▷ Initialize learnable parameters
2  $m \leftarrow 0$   ▷ Initial value of the first moment vector; same shape as  $\theta$ 
3  $v \leftarrow 0$   ▷ Initial value of the second moment vector; same shape as  $\theta$ 
4 for  $t = 1, \dots, T$  do
5   Sample batch of data  $B_t$ 
6    $g \leftarrow \nabla_{\theta} \ell(\theta; B_t)$   ▷ Compute the gradient of the loss
7    $\alpha_t \leftarrow \alpha \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}$   ▷ Compute adjusted  $\alpha$  for iteration  $t$ 
8    $\theta \leftarrow \theta - \alpha \lambda \theta$   ▷ Apply weight decay
9    $m \leftarrow \beta_1 m + (1 - \beta_1) g$   ▷ Update the first moment estimate
10   $v \leftarrow \beta_2 v + (1 - \beta_2) g^2$   ▷ Update the second moment estimate
11   $\theta \leftarrow \theta - \alpha_t \frac{m}{\sqrt{v + \epsilon}}$   ▷ Apply moment-adjusted weight updates
12 end for
```

算法 1: AdamW 优化器伪代码。

注意 t 从 1 开始。你现在将实现这个优化器。

问题 (adamw): 实现 AdamW (2 分)

交付物: 将 AdamW 优化器实现为 `torch.optim.Optimizer` 的子类。你的类应接受学习率 α (单位为 `__init__`) , 以及超参数 β 、 ϵ 和 λ 。为了帮助你保存状态, 基础 `Optimizer` 类提供了一个字典 `self.state`, 它将 `nn.Parameter` 对象映射到一个字典, 该字典存储你对该参数所需的任何信息 (对于 AdamW, 这将是动量估计)。实现 `[adapters.get_adamw_cls]` 并确保它通过

```
uv run pytest -k test_adamw.
```

问题 (adamw_accounting): 使用 AdamW 进行训练的资源计数 (2分)

让我们计算运行 AdamW 需要多少内存和计算量。假设我们对每个张量都使用 float32。

(a) 运行 AdamW 需要多少峰值内存? 请根据参数、激活值、梯度和优化器状态的内存使用情况分解你的答案。用 `batch_size` 和模型超参数 (`vocab_size`, `context_length`, `num_layers`, `d_model`, `num_heads`) 来表示你的答案。假设 $d_{\text{ff}} = \frac{8}{3} \times d_{\text{model}}$

为了简化起见, 在计算激活的内存使用时, 仅考虑以下组件:

- 变压器模块

RMSNorm(s) 多头自注意力子层: QKV 投影, QK^T 矩阵乘法, softmax, 加权求和值, 输出投影。按位置前馈 (SwiGLU): W_1, W_2 , 门控分支的 SiLU, 按元素相乘, W_3

- 最终 RMSNorm
- 输出嵌入
- 对 logits 的交叉熵

交付成果：每个参数、激活、梯度和优化器状态的代数表达式，以及总计。

(b) 为一个 GPT-2 XL 型模型实例化你的答案，以得到一个仅依赖于 `batch_size` 的表达式。你能使用的最大批量大小是多少，并且仍然适合 80GB 的内存？

交付物：一个看起来像 $a \cdot \text{batch_size} + b$ 的表达式用于数值 a, b ，以及一个表示最大批量大小的数字。

(c) 运行 AdamW 的一步需要多少 FLOPs？

成果物：一个代数表达式，并附简要说明。

(d) 模型 FLOPs 利用率 (MFU) 定义为观察到的吞吐量（每秒处理的 token 数）与硬件理论峰值 FLOP 吞吐量的比值 [A. Chowdhery 等, 2022]。NVIDIA H100 GPU 在 “float32”（实际上是 TensorFloat-3 2，实际上是 “bfloat19”）操作方面的理论峰值为 495 teraFLOP/s。假设你能够达到 50% 的 MFU，那么在单个 H100 上以批量大小为 1024 训练 GPT-2 XL 400K 步需要多长时间？按照 J. Kaplan 等 [25] 和 J. Hoffmann 等 [26] 的假设，反向传播的 FLOPs 是前向传播的两倍。

交付成果：培训所需的小时数，并附简要说明理由。

4.4 学习率调度

导致损失最快下降的学习率值在训练过程中通常会变化。在训练 Transformer 时，通常使用学习率 *schedule*，我们在开始时使用较大的学习率，以便更快地更新，然后随着模型训练缓慢衰减到较小的值⁸。在本次作业中，我们将实现用于训练 LLaMA 的余弦退火调度 [H. Touvron 等, 2023]。

调度器只是一个函数，它接收当前步骤 t 和其他相关参数（例如初始和最终学习率），并返回在步骤 t 更新梯度时使用的学习率。最简单的调度是常数函数，无论给定任何 t ，它都会返回相同的学习率。

余弦退火学习率调度采用 (i) 当前迭代 t ，(ii) 最大学习率 $\alpha_{\max}$ ，(iii) 最小（最终）学习率 $\alpha_{\min}$ ，(iv) *warm-up* 迭代的次数 T_w ，以及 (v) 余弦退火的最终迭代 T_c 。在第 t 次迭代的学习率定义为：

(热身) 如果 $t < T_w$ ，那么 $\alpha_t = \frac{t}{T_w} \alpha_{\max}$ 。 (余弦退火) 如果 $T_w \leq t \leq T_c$ ，那么 $\alpha_t = \alpha_{\min} + \frac{1}{2} \left(1 + \cos \left(\frac{t - T_w}{T_c - T_w} \pi \right) \right) (\alpha_{\max} - \alpha_{\min})$ 。 (退火后) 如果 $t > T_c$ ，那么 $\alpha_t = \alpha_{\min}$ 。

问题 (`learning_rate_schedule`): 实现带预热的余弦学习率调度 (1 分)

⁸It's sometimes common to use a schedule where the learning rate rises back up (restarts) to help get past local minima.

编写一个函数，该函数接受 t 、 $\alpha_{\max}$ 、 $\alpha_{\min}$ 、 T_w 和 T_c ，并根据上述定义的调度器返回学习率 α_t 。然后实现 `adapters.get_lr_cosine_schedule` 并确保它通过 `uv run pytest -k test_get_lr_cosine_schedule`。

4.5 梯度裁剪

在训练过程中，我们有时会遇到产生大梯度的训练样本，这可能会使训练不稳定。为缓解这一问题，实践中经常采用的一种技术是 *gradient clipping*。其思想是在每次反向传播后、进行优化器步骤之前，对梯度的范数施加限制。

给定所有参数的梯度 g ，我们计算其 ℓ_2 范数 $\|g\|_2$ 。如果该范数小于最大值 M ，则保持 g 不变；否则，将 g 缩放一个因子 $\frac{M}{\|g\|_2 + \varepsilon}$ （其中为了数值稳定性加入一个小的 ε ，例如 10^{-6} ）。注意，结果范数将略低于 M 。

问题 (**gradient_clipping**): 实现梯度裁剪 (1分)

编写一个实现梯度裁剪的函数。你的函数应接收一个参数列表和一个最大 ℓ_2 范数。它应当就地修改每个参数的梯度。使用 $\varepsilon = 10^{-6}$ (PyTorch 的默认值)。然后，实现适配器 `adapters.run_gradient_clipping` 并确保它通过 `uv run pytest -k test_gradient_clipping`。

5 训练循环

我们现在终于要把到目前为止构建的主要组件组合在一起：已分词的数据、模型和优化器。

5.1 数据加载器

分词后的数据（例如，你在 `tokenizer_experiments` 中准备的）是单一的标记序列 $x = (x_1, \dots, x_n)$ 。即使源数据可能由不同的文档组成（例如，不同的网页或源代码文件），常见的做法是将所有这些文档连接成一个单一的标记序列，在它们之间添加一个分隔符（例如 `<|endoftext|>` 标记）。

一个 *data loader* 将其转化为一个 *batches* 流，其中每个批次包含 B 个长度为 m 的序列，并与对应的下一个标记配对，长度也为 m 。例如，对于 $B = 1, m = 3$ ， $[(x_2, x_3, x_4), (x_3, x_4, x_5)]$ 将是一个潜在的批次。

以这种方式加载数据在多个方面简化了训练。首先，任何 $1 \leq i \leq n - m$ 都会提供一个有效的训练序列，因此采样训练序列非常简单。由于所有训练序列长度相同，因此无需对输入序列进行填充，这提高了硬件利用率（也通过增加批量大小 B ）。最后，我们也不需要加载整个数据集来采样训练数据，这使得处理可能无法全部加载到内存中的大型数据集变得容易。

问题 (**data_loading**): 实现数据加载 (2 分)

可交付成果：编写一个函数，该函数接收一个 numpy 数组 x （带有标记 ID 的整数数组）、一个 `batch_size`、一个 `context_length` 以及一个 PyTorch 设备字符串（例如 `'cpu'` 或 `'cuda:0'`），并返回一对张量：采样的输入序列和相应的下一个标记目标。两者

张量的形状应为 `(batch_size, context_length)`，包含令牌 ID，并且两者都应放置在请求的设备上。要将您的实现与我们提供的测试进行对比，您首先需要在 `[adapters.run_get_batch]` 实现测试适配器。然后，运行 `uv run pytest -k test_get_batch` 来测试您的实现。

低资源技巧：在 CPU 或 Apple Silicon 上加载数据

如果你计划在 CPU 或 Apple Silicon 上训练你的语言模型，你需要将数据移动到正确的设备上（同样，之后你也应该对模型使用相同的设备）。

如果你使用的是 CPU，你可以使用 `'cpu'` 设备字符串；如果是苹果硅（M* 芯片），你可以使用 `'mps'` 设备字符串。

有关 MPS 的更多信息，请查看以下资源：

- <https://docs.pytorch.org/docs/stable/mps.html>
- <https://docs.pytorch.org/docs/stable/notes/mps.html>
- <https://developer.apple.com/documentation/metalperformanceshaders>

如果数据集太大而无法加载到内存中怎么办？我们可以使用一个名为 `mmap` 的 Unix 系统调用，它将磁盘上的文件映射到虚拟内存，并在访问该内存位置时懒加载文件内容。因此，你可以“假装”将整个数据集都加载到内存中。Numpy 通过 `np.memmap`（或者如果你最初使用 `np.save` 保存数组，则使用标志 `mmap_mode='r'` 到 `np.load`）来实现这一点，它将返回一个类似 numpy 数组的对象，该对象会在你访问条目时按需加载它们。在训练期间从数据集中（即 numpy 数组）采样时，请确保以内存映射模式加载数据集（通过 `np.memmap` 或标志 `mmap_mode='r'` 到 `np.load`，取决于你保存数组的方式）。还要确保指定一个与所加载数组匹配的 `dtype`。显式验证内存映射数据是否正确可能会很有帮助（例如，不包含超出预期词汇量的值）。

5.2 检查点

除了加载数据，我们还需要在训练过程中保存模型。在运行任务时，我们通常希望能够恢复中途停止的训练（例如，由于任务超时、机器故障等）。即使一切顺利，我们可能也希望以后能够访问中间模型（例如，为了事后研究训练动态，从训练不同阶段的模型中取样等）。

检查点应包含我们恢复训练所需的所有状态。当然，我们至少希望能够恢复模型权重。如果使用有状态优化器（例如 AdamW），我们还需要保存优化器的状态（例如，对于 AdamW，是动量估计值）。最后，为了恢复学习率调度，我们需要知道停止时的迭代次数。PyTorch 使保存所有这些变得容易：每个 `nn.Module` 都有一个 `state_dict()` 方法，该方法返回包含所有可学习权重的字典；我们可以稍后使用对应的 `load_state_dict()` 方法恢复这些权重。任何 `torch.optim.Optimizer` 也同样适用。最后，`torch.save(obj, dest)` 可以将对象（例如包含张量作为某些值的字典，也可以是普通 Python 对象如整数）写入文件（路径）或类文件对象，然后可以使用 `torch.load(src)` 将其加载回内存。

问题 (checkpointing): 实现模型检查点 (1分)

实现以下两个函数以加载和保存检查点:

def save_checkpoint(model, optimizer, iteration, out)应该将模型、优化器和迭代的所有状态转储到类似文件的对象 out 中。你可以使用模型和优化器的 `state_dict` 方法来获取它们相关的状态, 并使用 `torch.save(obj, out)` 将 obj 转储到 out 中 (PyTorch 这里支持路径或类似文件的对象)。一个典型的选择是让 obj 成为一个字典, 但你可以使用任何你想要的格式, 只要你之后能够加载你的检查点。

此函数期望以下参数:

```
model: torch.nn.Module
optimizer: torch.optim.Optimizer
iteration: int
out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
```

def load_checkpoint(src, model, optimizer)应该从 src (路径或类文件对象) 加载一个检查点, 然后从该检查点恢复模型和优化器状态。你的函数应该返回保存到检查点的迭代次数。你可以使用 `torch.load(src)` 来恢复在你的 `save_checkpoint` 实现中保存的内容, 并使用模型和优化器中的 `load_state_dict` 方法将它们恢复到之前的状态。

此函数期望以下参数:

```
src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]
model: torch.nn.Module
optimizer: torch.optim.Optimizer
```

实现 `[adapters.run_save_checkpoint]` 和 `[adapters.run_load_checkpoint]` 适配器, 并确保它们通过 `uv run pytest -k test_checkpointing`。

5.3 训练循环

现在, 终于到了将你实现的所有组件整合到你的主训练脚本中的时候了。让启动不同超参数的训练运行变得容易是值得的 (例如, 通过将它们作为命令行参数), 因为你以后会为了研究不同选择如何影响训练而多次进行这些操作。

P 问题 (training_together): 把它组合起来 (4 分) nts)

交付物: 编写一个脚本, 该脚本运行训练循环以训练模型使用用户提供的输入。特别是, 我们建议您的训练脚本至少允许以下功能:

- 能够配置和控制各种模型和优化器的超参数。
- 使用 `np.memmap` 高效内存加载大型训练和验证数据集。
- 将检查点序列化到用户提供的路径。

- 定期记录训练和验证性能（例如，输出到控制台和/或像 Weights and Biases 这样的外部服务）⁹

6 生成文本

既然我们可以训练模型，最后需要的部分是能够从我们的模型生成文本。回想一下，语言模型接收一个（可能是批处理的）长度为 `sequence_length` 的整数序列，并生成一个大小为 $(\text{sequence_length}, \text{vocab_size})$ 的矩阵，其中序列的每个元素都是预测该位置之后下一个标记的概率分布。我们现在将编写一些函数，将其转换为生成新序列的采样方案。

软最大化

按照标准惯例，语言模型的输出是最后一层线性层的输出（“logits”），因此我们必须通过之前看到的 *softmax* 操作将其转化为归一化的概率。

在方程 10 中。

解码

为了从我们的模型生成文本（解码），我们将向模型提供一系列前缀标记（“提示”），并要求它对词汇表生成下一个序列标记的概率分布。然后，我们将从这个词汇表项的分布中进行采样，以确定下一个输出标记。

具体来说，解码过程的一步应该接收一个序列 $x_{1\dots t}$ 并通过以下方程返回一个标记 x_{t+1} ，

$$P(x_{t+1} = i \mid x_{1\dots t}) = \frac{\exp(v_i)}{\sum_j \exp(v_j)} \quad (21)$$

$$v = \text{TransformerLM}(x_{1\dots t})_t \in \mathbb{R}^{\text{vocab_size}} \quad (22)$$

其中 TransformerLM 是我们的模型，它接受长度为 `sequence_length` 的序列作为输入，并生成大小为 $(\text{sequence_length}, \text{vocab_size})$ 的矩阵，我们取该矩阵的最后一个元素，因为我们正在寻找在第 t 个位置的下一个标记预测。

这给了我们一个基本的解码器，通过反复从这些一步条件分布中采样（将我们之前生成的输出标记附加到下一个解码时间步的输入中），直到我们生成序列结束标记 `<|endoftext|>`（或用户指定的最大生成标记数量）。

解码器技巧

我们将尝试使用小模型，而小模型有时可能生成质量很低的文本。两个简单的解码器技巧可以帮助解决这些问题。首先，在 *temperature scaling* 中，我们用一个温度参数 τ 修改了 softmax，其中新的 softmax 是

$$\text{softmax}(v, \tau)_i = \frac{\exp(v_i/\tau)}{\sum_{j=1}^{\text{vocab_size}} \exp(v_j/\tau)} \quad (23)$$

注意设置 $\tau \rightarrow 0$ 会使 v 的最大元素占主导地位，并且 softmax 的输出会变成集中在该最大元素上的 one-hot 向量。

⁹wandb.ai

其次，另一种技巧是 *nucleus* 或 *top-p* 采样，我们通过截断低概率的标记来修改采样分布。设 q 为我们从大小为 `vocab_size` 的（温度缩放）softmax 得到的概率分布。使用超参数 p 的核心采样根据以下公式生成下一个标记

$$P(x_{t+1} = i \mid q) = \begin{cases} \frac{q_i}{\sum_{j \in V(p)} q_j} & \text{if } i \in V(p) \\ 0 & \text{otherwise} \end{cases} \quad (24)$$

其中 $V(p)$ 是满足 $\sum_{j \in V(p)} q_j \geq p$ 的 *smallest* 指数集合。你可以通过先按大小对概率分布 q 进行排序，然后选择最大的词汇元素，直到达到目标水平 p ，轻松计算这个量。

问题 (decoding): 解码 (3 分)

交付物：实现一个函数从你的语言模型中解码。我们建议你支持以下功能：

- 为用户提供的提示生成完成内容（即，输入一些 $x_{1:t}$ 并采样一个完成，直到遇到 `<|endoftext|>` 令牌）。
- 允许用户控制生成的最大标记数。
- 给定一个期望的温度值，在采样之前对预测的下一个标记分布应用 softmax 温度缩放。
- Top- p 采样 ([A. Holtzman 等人, 2020] 也称为核采样)，给定用户指定的阈值。

7 个实验

现在是把所有内容整合在一起，并在预训练数据集上训练（小型）语言模型的时候了。

7.1 如何进行实验及交付物

理解Transformer架构组件背后原理的最佳方式是实际修改它并自己运行它。实践经验是无法替代的。

为此，能够快速、一致地进行实验并记录所做的工作非常重要。为了快速实验，我们将对一个小规模模型（总参数约为1700万）和简单的数据集（TinyStories）进行许多实验。为了做到一致性，你将以系统的方式消融组件和改变超参数；为了记录，我们会要求你提交实验日志，以及与每个实验相关的学习曲线。

为了能够提交损失曲线，请确保持定期评估验证损失，并记录步数和实际时间。你可能会发现像 `Weights and Biases` 这样的日志记录工具很有帮助。

问题 (experiment_log): 实验记录 (3 分)

对于你的训练和评估代码，创建实验跟踪基础设施，使你能够跟踪实验以及相对于梯度步数和实际时间的损失曲线。

交付物：用于你的实验的日志基础设施代码，以及本节下面作业问题的实验日志（记录你尝试过的所有内容的文档）。

7.2 小故事

我们将从一个非常简单的数据集开始（TinyStories；R. Eldan 等 [1]），模型将在这里快速训练，我们可以看到一些有趣的行为。获取此数据集的说明在第 1 节。下面是该数据集的示例。

示例 (**tinystories_example**): 来自 TinyStories 的一个例子

很久很久以前，有一个小男孩叫本。本喜欢探索他周围的世界。他看到了许多令人惊叹的东西，比如商店里展示的美丽花瓶。有一天，本正在商店里走着，偶然发现了一个非常特别的花瓶。当本看到它时，他惊讶极了！他说：“哇，这真是一个非常棒的花瓶！我可以买吗？”店主笑着说：“当然可以。你可以把它带回家，向所有朋友展示它有多么棒！”于是本把花瓶带回了家，他非常骄傲！他叫来朋友们，向他们展示这个令人惊叹的花瓶。所有朋友都觉得花瓶很美，不敢相信本是多么幸运。就这样，本在商店里发现了一个令人惊叹的花瓶！

7.2.1 超参数调优

我们将告诉你一些非常基本的超参数以开始，并要求你为其他参数找到一些效果良好的设置。

词汇量 10000。典型的词汇量范围在几十到几十万之间。你应该尝试变化这个数值，看看词汇量和模型行为如何变化。

上下文长度为256。像TinyStories这样简单的数据集可能不需要很长的序列长度，但对于后来的OpenWebText数据，你可能想要调整这个参数。尝试调整它，并观察对每次迭代运行时间和最终困惑度的影响。

d_model512。这比许多小型 Transformer 论文中使用的 768 维度略小，但这会让事情更快。

d_ff1344。这大约是 $\frac{8}{3}d_{\text{model}}$ ，同时是64的倍数，这对GPU性能有好处。

RoPE theta 参数 Θ 10000。

层数和注意力头数 4 层，16 个头。总体而言，这将提供约 1700 万个非嵌入参数，这是一个相当小的 Transformer。

处理的总令牌数 327,680,000（您的批量大小 $\times$ 总步数 $\times$ 上下文长度应大致等于此值）。

你应该通过反复试验来找到以下其他超参数的良好默认值：learning rate、learning rate warmup、other AdamW hyperparameters、 $(\beta_1, \beta_2, \epsilon)$ 和 weight decay。你可以在 D. P. Kingma 等人 [22] 中找到这些超参数的一些典型选择。

7.2.2 综合起来

现在你可以通过获取一个训练好的 BPE 分词器、对训练数据集进行分词，并在你编写的训练循环中运行它，将所有内容整合在一起。重要说明：如果你的实现是正确且高效的，上述超参数在 1 个 B200 上的运行时间大约应为 20–30 分钟。

GPU。如果你的运行时间长得多，请检查并确保你的数据加载、检查点保存或验证损失代码没有成为运行时间的瓶颈，并且你的实现已经正确批处理。

7.2.3 调试模型架构的技巧和窍门

我们强烈建议熟悉你IDE内置的调试器（例如，VSCode/Zed），这相比用打印语句调试会节省你的时间。如果你使用文本编辑器，你可以使用类似 `ipdb` 的工具。在调试模型架构时，其他一些好的做法包括：

- 在开发任何神经网络架构时，一个常见的第一步是对单个小批量进行过拟合。如果你的实现是正确的，你应该能够快速将训练损失降到接近零。
- 在各种模型组件中设置调试断点，并检查中间张量的形状，以确保它们符合你的预期。
- 监控激活值、模型权重和梯度的规范，以确保它们不会爆炸或消失。

问题 (**learning_rate**)： 调整学习率 (2 B200 小时) (3 分)

学习率是最需要调整的重要超参数之一。基于你训练的基础模型，回答以下问题：

(a) 对学习率进行超参数扫描，并报告最终损失（如果优化器发散，则注明发散情况）。

交付成果：与多种学习率相关的学习曲线。解释你的超参数搜索策略。

交付成果：在 TinyStories 上每个 token 验证损失不超过 1.45 的模型

低资源提示：在 CPU 或 Apple Silicon 上训练几步

如果你正在运行 `cpu` 或 `mps`，你应该将处理的总令牌数减少到 40,000,000，这将足以生成相当流畅的文本。你也可以将目标验证损失从 1.45 提高到 2.00。

在 M4 Max 芯片和 36 GB 内存上以调优的学习率运行我们的解决方案代码时，我们使用批量大小 $\times$ ，总步数 $\times$ ，上下文长度 $= 32 \times 5000 \times 256 =$ ，40,960,000 个标记，在 `cpu` 上耗时 1 小时 22 分钟，在 `mps` 上耗时 36 分钟。在第 5000 步时，我们达到了 1.80 的验证损失。

一些额外的提示：

- 在使用 N 训练步骤时，我们建议调整余弦学习率衰减计划，以在第 N 步精确终止其衰减（即达到最小学习率）。
- 使用 `mps` 时，不要使用 TF32 内核，即不要设置

```
torch.set_float32_matmul_precision('high')
```

就像你可能对 `cuda` 设备所做的那样。我们尝试在 `mps` (torch 版本 2.9.0) 上启用 TF32 内核，发现后端有时会使用默默损坏的内核，这会导致训练不稳定。

- 您可以通过使用 `torch.compile` 对模型进行 JIT 编译来加快训练速度。具体来说：在 `cpu` 上，使用 `model = torch.compile(model)` 编译您的模型。在 `mps` 上，您可以使用 `model = torch.compile(model, backend="aot_eager")` 对反向传播进行一定程度的优化。截至 `torch` 版本 2.9.0，Inductor 编译在 `mps` 上不受支持。

(b) 民间智慧认为，最佳学习率是在“稳定边缘”。研究学习率发散的点如何与你的最佳学习率相关。

交付成果：学习率逐步增加的学习曲线，其中至少包含一次发散的运行，并分析这与收敛速度的关系。

现在让我们改变批量大小，看看训练会发生什么。批量大小很重要——它们可以让我们的 GPU 通过进行更大的矩阵乘法获得更高的效率，但我们总是希望批量大小很大，这是真的吗？让我们做一些实验来找出答案。

问题 (**batch_size_experiment**)：批量大小变化 (1 B200 小时) (1 分)

将你的批量大小从 1 变化到 GPU 内存的极限。至少尝试几个中间的批量大小，包括典型的大小如 64 和 128。

交付物：不同批量大小运行的学习曲线。如有必要，应再次优化学习率。

交付成果：几句话讨论你关于批量大小及其对训练影响的发现。

有了你的解码器，我们现在可以生成文本了！我们将从模型中生成文本，看看它有多好。作为参考，你应该得到至少和下面示例一样好的输出。

示例 (**ts_generate_example**)：来自 TinyStories 语言模型的示例输出

从前，有一个漂亮的女孩叫莉莉。她喜欢吃口香糖，尤其是那种大的黑色口香糖。有一天，莉莉的妈妈让她帮忙做晚饭。莉莉非常兴奋！她喜欢帮妈妈。莉莉的妈妈做了一大锅汤作为晚餐。莉莉非常高兴，说：“谢谢你，妈妈！我爱你。”她帮妈妈把汤倒进一个大碗里。晚餐后，莉莉的妈妈又做了一些美味的汤。莉莉非常喜欢，她说：“谢谢你，妈妈！这汤太好喝了！”她妈妈微笑着说：“我很高兴你喜欢，莉莉。”他们做完饭后继续一起做饭。故事结束。

低资源技巧：在 CPU 或 Apple Silicon 上生成文本

如果你使用处理了4000万标记的低资源配置，你应该会看到生成的内容仍然类似英语，但流畅度不如上面。例如，我们使用在4000万标记上训练的TinyStories语言模型的示例输出如下：

很久很久以前，有一个名叫苏的小女孩。苏有一颗她非常喜欢的牙齿。那是她最好的头。有一天，苏去散步，遇到了一只瓢虫！它们成了好朋友，一起在小路上玩耍。

“嘿，波莉！我们出去吧！”蒂姆说。苏看了看天空，发现很难找到一种闪亮的舞蹈方式。她笑着同意帮助说话的人！

当苏看着天空移动时，那是什么。她

这是准确的问题陈述以及我们所要求的内容：

问题 (**generate**): 生成文本 (1 分)

使用你的解码器和经过训练的检查点，报告你的模型生成的文本。你可能需要调整解码器参数（温度、top-p 等）以获得流畅的输出。

交付物：至少256个标记的文本转储（或直到第一个 `<|endoftext|>` 标记），以及对该输出流畅性的简要评论和至少两个影响该输出好坏的因素。

7.3 消融实验与架构修改

理解 Transformer 的最佳方式是实际修改它并观察它的行为。我们现在将进行一些简单的消融和修改。

消融 1: 层归一化

人们常说层归一化对 Transformer 训练的稳定性非常重要。但也许我们想冒险。让我们从每个 Transformer 模块中移除 RMSNorm，看看会发生什么。

问题 (**layer_norm_ablation**): 移除 RMSNorm 并训练 (0.5 B200 小时) (1 分)

从你的 Transformer 中移除所有 RMSNorm，并进行训练。在之前的最优学习率下会发生什么？使用较低的学习率可以获得稳定性吗？

交付成果：在移除 RMSNorm 并进行训练时的学习曲线，以及最佳学习率的学习曲线。

可交付成果：关于RMSNorm影响的几句评论。

现在让我们来研究另一种乍看起来似乎是任意的层归一化选择。Pre-norm Transformer 块被定义为

$$z = x + \text{MultiHeadSelfAttention}(\text{RMSNorm}(x)) \quad (25)$$

$$y = z + \text{FFN}(\text{RMSNorm}(z)). \quad (26)$$

这是对原始 Transformer 架构为数不多的“共识”修改之一，它使用了 *post-norm* 方法作为

$$z = \text{RMSNorm}(x + \text{MultiHeadSelfAttention}(x)) \quad (27)$$

$$y = \text{RMSNorm}(z + \text{FFN}(z)). \quad (28)$$

让我们回到 *post-norm* 方法，看看会发生什么。

问题 (**pre_norm_ablation**): 实现后归一化并训练 (0.5 B200 小时) (1 分)

将你的前规范 Transformer 实现修改为后规范实现。使用后规范模型进行训练，看看会发生什么。

交付成果：后归一化 Transformer 的学习曲线，与前归一化 Transformer 相比。

我们看到层归一化对 Transformer 的行为有重大影响，甚至层归一化的位置也很重要。

消融 2：位置嵌入

接下来我们将研究位置嵌入对模型性能的影响。具体来说，我们将比较基础模型（使用 RoPE）与完全不使用位置嵌入（NoPE）的情况。事实证明，仅有解码器的变压器，即我们实现的带有因果掩码的模型，理论上可以在不显式提供位置嵌入的情况下推断相对或绝对的位置信息 [Y.-H. H. Tsai 等, 2019; A. Kazemnejad 等, 2023]。现在我们将通过实验证明 NoPE 与 RoPE 的性能对比。

问题 (**no_pos_emb**): 实现 NoPE (0.5 B200 小时) (1 分)

修改你的 Transformer 实现，使用 RoPE 完全移除位置信息嵌入，看看会发生什么。

交付成果：比较 RoPE 和 NoPE 性能的学习曲线。

消融实验3：SwiGLU 与 SiLU

接下来，我们将遵循 [N. Shazeer \[20\]](#) 的做法，测试前馈网络中门控的重要性，通过比较 SwiGLU 前馈网络与使用 SiLU 激活但没有门控线性单元 (GLU) 的前馈网络的性能：

$$\text{FFN}_{\text{SiLU}}(x) = W_2 \text{SiLU}(W_1 x). \quad (29)$$

请记住，在我们的 SwiGLU 实现中，我们将内部前馈层的维度设置为大约 $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$ （同时确保 $d_{\text{ff}} \bmod 64 = 0$ ，以利用 GPU 张量核心）。在这个消融基线中，你的 FFN_{SiLU} 实现应改为设置 $d_{\text{ff}} = 4 \times d_{\text{model}}$ ，以大致匹配默认 SwiGLU 前馈网络的参数数量（后者有三个而不是两个权重矩阵）。

问题 (**swiglu_ablation**): SwiGLU 与 SiLU (0.5 B200 小时) (1 分)

交付成果：一个学习曲线，用于比较 SwiGLU 和 SiLU 前馈网络的性能，其参数数量大致相当。

交付成果：几句话讨论你的发现。

低资源提示：GPU 资源有限的在线学生应在 TinyStories 上测试修改

在作业的其余部分，我们将转向一个更大规模、更嘈杂的网络数据集（OpenWebText），尝试架构修改，并（可选地）向课程排行榜提交。

在 OpenWebText 上训练语言模型达到流利需要很长时间，因此我们建议 GPU 资源有限的在线学生继续在 TinyStories 上测试修改（使用验证损失作为评估性能的指标）。

7.4 在 OpenWebText 上运行

我们现在将转向一个更标准的预训练数据集，该数据集由网络抓取创建。还提供了 OpenWebText [A. Gokaslan 等, 2019]的一个小样本作为单个文本文件：有关如何访问此文件，请参见第 1 节。

这是来自 OpenWebText 的一个示例。注意文本更加真实、复杂且多样化。你可能想浏览一下训练数据集，以了解网络爬取语料库的训练数据是什么样的。

示例 (owt_example): 来自 OWT 的一个例子

棒球前景杂志技术总监哈里·帕夫利迪斯在雇佣乔纳森·贾奇时冒了风险。

帕夫利迪斯知道，正如艾伦·施瓦茨在《数字游戏》中写的那样，“没有美国文化的哪个角落比棒球运动员的表现更精确地被计算，更热情地被量化。”只需几次点击，你就可以发现诺亚·辛德加德的速球在到达本垒板的过程中每分钟旋转超过2100次，纳尔逊·克鲁兹在2016年合格击球手中拥有全场最高的平均出球速度，以及其他无数看起来像是从电子游戏或科幻小说中撕下来的趣闻。不断上升的数据海洋赋予了棒球文化中一个日益重要的角色：分析爱好者。

这种赋权伴随着更多的审查——不仅是对测量的审查，还有对测量背后的人员和出版物的审查。在《棒球前景》(Baseball Prospectus) 中，帕夫利迪斯(Pavlidis) 对定量不完美所引发的反弹了如指掌。他也知道该网站的捕手指标需要重新设计，而且这项工作需要一个有学识的头脑——一个能够处理复杂统计建模问题的人来完成。

“他让我们很害怕。” 哈里·帕夫利迪斯

帕夫利迪斯有一种直觉，认为法官“明白了”，这是基于后者的写作以及他们在一次网站赞助的棒球场活动上的互动。[...]

注意：您可能需要重新调整您的超参数，例如学习率或批量大小，以进行此实验。

问题 (main_experiment): 在 OWT 上实验 (2 B200 小时) (2 分)

使用与TinyStories相同的模型架构和总训练迭代次数，在OpenWebText上训练你的语言模型。这个模型表现如何？

交付成果：你在 OpenWebText 上的语言模型学习曲线。描述与 TinyStories 的损失差异——我们应如何解释这些损失？

交付物：来自 OpenWebText 语言模型的生成文本，格式与 TinyStories 输出相同。这段文本的流畅度如何？即使我们使用与 TinyStories 相同的模型和计算预算，输出质量为什么会更差？

7.5 你自己的修改版 + 排行榜

恭喜你达到这一阶段。你几乎完成了！现在你将尝试改进 Transformer 架构，并看看你的超参数和架构与班上其他学生相比如何。

排行榜规则

除了以下内容外没有其他限制：

运行时间：您的提交在 B200 上最多可以运行 45 分钟。如果您使用 SLURM 或 Modal，可能需要在提交脚本中强制执行这一限制。

数据：您只能使用我们提供的 OpenWebText 训练数据集。

否则，你可以自由地做任何你内心渴望的事情。

如果你正在寻找一些实现的想法，你可以查看以下一些资源：

- 最先进的开源大型语言模型系列，例如 Llama 3 [A. Grattafiori 等, 2024] 或 Qwen 2.5 [A. Yang 等, 2024]
- NanoGPT 速通仓库 (github.com/KellerJordan/modded-nanogpt)，社区成员在这里发布了许多有趣的小规模语言模型预训练“速通”的修改。例如，一种流行的修改方法可追溯到最初的 Transformer 论文，即将输入和输出嵌入的权重绑定在一起（参见 A. Vaswani 等人 [8]（第 3.4 节）和 A. Chowdhery 等人 [16]（第 2 节））。如果你尝试权重绑定，可能需要降低嵌入/语言模型头初始化的标准差。

在尝试完整的 45 分钟运行之前，你会想先在 OpenWebText 的一个小子集或 TinyStories 上测试这些。

需要注意的是，我们确实注意到，在这个排行榜上工作良好的一些修改，可能无法推广到大规模的预训练中。我们将在课程的规模法则单元中进一步探讨这个想法。

问题 (leaderboard): 排行榜 (10 B200 小时) (6 分)

你将按照上述排行榜规则训练一个模型，目标是在 0.75 B200 小时内最小化你的语言模型的验证损失。

交付物：记录的最终验证损失、一个关联的学习曲线，该曲线清楚地显示了小于45分钟的实时时间（wall-clock-time）x轴，以及你所做工作的描述。我们期望排行榜提交至少要超过5.0损失的简单基线。在此处向排行榜提交：github.com/stanford-cs336/assignment1-basics-leaderboard。

参考文献

- [1] R. Eldan 和 Y. Li, “TinyStories: 语言模型能多小还能够说连贯的英文?” 2023.
- [2] A. Gokaslan, V. Cohen, E. Pavlick, 和 S. Tellex, “OpenWebText 语料库。” 2019。
- [3] R. Sennrich, B. Haddow, 和 A. Birch, “使用子词单元进行罕见词的神经机器翻译,” 发表于 *Proc. of ACL*, 2016。
- [4] C. Wang, K. Cho, 和 J. Gu, “基于字节级子词的神经机器翻译。” 2019.
- [5] P. Gage, “一种用于数据压缩的新算法,” *C Users Journal*, 第12卷, 第2期, 页23–38, 1994年2月。
- [6] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei 和 I. Sutskever, “语言模型是无监督多任务学习者。” 2019. [7] A. Radford, K. Narasimhan, T. Salimans 和 I. Sutskever, “通过生成式预训练改进语言理解。” 2018。
- [8] A. Vaswani *et al.*, “注意力机制就是你所需要的一切”, 发表于 *Proc. of NeurIPS*, 2017。[9] T. Q. Nguyen 和 J. Salazar, “无痛变压器: 改进自注意力的归一化”, 发表于 *Proc. of IWSWLT*, 2019。[10] R. Xiong *et al.*, “变压器架构中的层归一化”, 发表于 *Proc. of ICML*, 2020。[11] J. L. Ba, J. R. Kiros 和 G. E. Hinton, “层归一化。” 2016。[12] H. Touvron *et al.*, “LLaMA: 开放且高效的基础语言模型。” 2023。
- [13] B. Zhang 和 R. Sennrich, “均方根层归一化”, 发表于 *Proc. of NeurIPS*, 2019。[14] A. Grattafiori *et al.*, “Llama 3 模型族。” [在线]. 可访问: <https://arxiv.org/abs/2407.21783> [15] A. Yang *et al.*, “Qwen2.5 技术报告”, *arXiv preprint arXiv:2412.15115*, 2024。[16] A. Chowdhery *et al.*, “PaL”

[21] J. Su, Y. Lu, S. Pan, B. Wen, 和 Y. Liu, “RoFormer: 带旋转位置嵌入的增强型 Transformer。” 2021. [22] D. P. Kingma 和 J. Ba, “Adam: 一种随机优化方法,” 发表于 *Proc. of ICLR*, 2015. [23] I. Loshchilov 和 F. Hutter, “解耦权重衰减正则化,” 发表于 *Proc. of ICLR*, 2019. [24] T. B. Brown *et al.*, “语言模型是少样本学习者,” 发表于 *Proc. of NeurIPS*, 2020. [25] J. Kaplan *et al.*, “神经语言模型的缩放定律。” 2020. [26] J. Hoffmann *et al.*, “训练计算最优的大型语言模型。” 2022. [27] A. Holtzman, J. Buys, L. Du, M. Forbes, 和 Y. C. hoi, “神经文本退化的奇怪案例,” 发表于 *Proc. of ICLR*, 2020. [28] Y.-H. H. Tsai, S. Bai, M. Yamada, L.-P. Morency, 和 R. Salakhutdinov, “Transformer 解剖: 通过核视角对 Transformer 注意力的统一理解,” 发表于 *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, 和 X. Wan, 编辑, 中国香港: 计算语言学协会, 2019年11月, 第4344–4353页. doi: 10.186

